

Raw signal segmentation for estimating RNA modification from Nanopore direct RNA sequencing data

Reviewed Preprint

v3 • November 10, 2025

Revised by authors

Reviewed Preprint

v2 • October 15, 2025

Reviewed Preprint

v1 • March 6, 2025

Guangzhao Cheng, Aki Vehtari, Lu Cheng ✉

Department of Computer Science, Aalto University, Espoo, Finland • Institute of Biomedicine, University of Eastern Finland, Kuopio, Finland

https://en.wikipedia.org/wiki/Open_access

Copyright information

eLife Assessment

This study presents SegPore, a **valuable** new method for processing direct RNA nanopore sequencing data, which improves the segmentation of raw signals into individual bases and boosts the accuracy of modified base detection. The evidence presented to benchmark SegPore is **solid**, and the authors provide a fully documented implementation of the method. SegPore will be of particular interest to researchers studying RNA modifications.

<https://doi.org/10.7554/eLife.104618.3.sa4>

Abstract

Estimating RNA modifications from Nanopore direct RNA sequencing data is a critical task for the RNA research community. However, current computational methods often fail to deliver satisfactory results due to inaccurate segmentation of the raw signal. We have developed a new method, SegPore, which leverages a molecular jiggling translocation hypothesis to improve raw signal segmentation. SegPore is a pure white-box model with enhanced interpretability, significantly reducing structured noise in the raw signal. We demonstrate that SegPore outperforms state-of-the-art methods, such as Nanopolish and Tombo, in raw signal segmentation across three large benchmark datasets. Moreover, the improved signal segmentation achieved by SegPore enables SegPore+m6Anet to deliver state-of-the-art performance in site-level m6A identification. Additionally, SegPore surpasses baseline methods like CHEUI in single-molecule level m6A identification.

Introduction

RNA modifications play important roles in different diseases, such as Acute Myeloid Leukemia (1) and Fragile X Syndrome (2), as well as fundamental biological processes like cell differentiation (3,4) and immune response (5). To date, researchers have identified over 150 different types of RNA modifications (6-9), highlighting the complexity and diversity of

RNA regulation. These modifications are essential for proper RNA function, including maintaining secondary structures and facilitating accurate protein synthesis, such as the role of tRNA modifications in decoding mRNA codons (10 [↗](#)).

The practical applications of RNA modifications are far-reaching. For instance, N1-methylpseudouridine (m1Ψ) has been used to enhance the efficacy of COVID-19 mRNA vaccines, highlighting their therapeutic potential (11 [↗](#)). However, identifying and characterizing RNA modifications presents significant challenges due to the limitations of existing experimental techniques.

Traditional methods for RNA modification detection, such as MeRIP-Seq (12 [↗](#)), miCLIP (13 [↗](#)), and m6ACE-Seq (14 [↗](#)), rely on immunoprecipitation techniques that use modification-specific antibodies. While effective, these methods have several drawbacks. First, each method requires a separate antibody for each RNA modification, making it difficult to study multiple modifications simultaneously. Additionally, these techniques can only infer modification locations from next-generation sequencing (NGS) data, rather than measuring the modifications directly. As a result, they struggle to provide single-molecule resolution, limiting their accuracy and scope.

Recent advancements in direct RNA sequencing (DRS) by Oxford Nanopore Technologies (ONT) offer a promising alternative. DRS allows for the direct measurement of electrical currents as RNA molecules translocate through a nanopore, providing a potential avenue for detecting RNA modifications at the single-molecule level. Two versions of the direct RNA sequencing (DRS) kits are available: RNA002 and RNA004. Unless otherwise specified, this study focuses on RNA002 data. In this technique, five nucleotides (5-mers) reside in the nanopore at a time, and each 5-mer generates a characteristic current signal based on its unique sequence and chemical properties (16 [↗](#)). By analyzing these signals, it is possible to infer the original RNA sequence and detect modifications like N6-methyladenosine (m6A).

The general workflow of Nanopore direct RNA sequencing (DRS) data analysis is as follows. First, the raw electrical signal from a read is basecalled using tools such as Guppy or Dorado (<https://github.com/nanoporetech> [↗](#)), which produce the nucleotide sequence of the RNA molecule. However, these basecalled sequences do not include the precise start and end positions of each ribonucleotide (or k-mer) in the signal. Because basecalling errors are common, the sequences are typically mapped to a reference genome or transcriptome using minimap2 (15 [↗](#)) to recover the correct reference sequence. Next, tools such as Nanopolish (16 [↗](#), 17 [↗](#)) and Tombo (18 [↗](#)) align the raw signal to the reference sequence to determine which portion of the signal corresponds to each k-mer. We define this process as the segmentation and alignment task (abbreviated as the segmentation task), which is referred to as “eventalign” in Nanopolish. Based on this alignment, Nanopolish extracts various features—such as the start and end positions, mean, and standard deviation of the signal segment corresponding to a k-mer. This signal segment or its derived features is referred to as an “event” in Nanopolish. The resulting events serve as input for downstream RNA modification detection tools such as m6Anet (19 [↗](#)) and CHEUI (20 [↗](#)).

However, significant computational challenges remain. Segmenting the raw current signal into distinct 5-mers and distinguishing between normal nucleotides and their modified counterparts is a complex task. Current methods, such as Nanopolish, employ change-point detection methods to segment the signal and use dynamic programming methods and Hidden Markov Models (HMM) to align the derived segments to the reference sequence, but they are prone to noise and inaccuracies, which degrade performance in downstream tasks like RNA modification prediction. The root of this issue lies in the fact that these methods do not accurately model the physical process of Nanopore sequencing, particularly the dynamics of the motor protein that drives RNA through the pore. As a result, the segmentation process is not well-aligned with the actual translocation mechanics, leading to signal noise that hinders precise modification detection.

SegPore is a novel tool for direct RNA sequencing (DRS) signal segmentation and alignment, designed to overcome key limitations of existing approaches. By explicitly modeling motor protein dynamics during RNA translocation with a Hierarchical Hidden Markov Model (HHMM), SegPore segments the raw signal into small, biologically meaningful fragments, each corresponding to a k -mer sub-state, which substantially reduces noise and improves segmentation accuracy. After segmentation, these fragments are aligned to the reference sequence and concatenated into larger events, analogous to Nanopolish's "eventalign" output, which serve as the foundation for downstream analyses. Moreover, the "eventalign" results produced by SegPore enhance interpretability in RNA modification estimation. While deep learning-based tools such as m6Anet classify RNA modifications using complex, non-transparent features (see Supplementary Fig. S5), SegPore employs a simple Gaussian Mixture Model (GMM) to distinguish modified from unmodified nucleotides based on baseline current levels. This transparent modeling approach improves confidence in the predictions and makes SegPore particularly well-suited for biological applications where interpretability is essential.

By introducing SegPore, we bridge the gaps left by traditional tools. SegPore utilizes a hierarchical hidden Markov model (HHMM) for more precise segmentation and combines it with signal alignment and a GMM for RNA modification prediction, offering both greater accuracy and interpretability. This integrated approach enables robust m6A modification detection at the single-molecule level, facilitating reliable, transparent predictions for both site-specific and molecule-wide m6A modifications.

Materials and methods

SegPore workflow

Workflow overview

An overview of SegPore is illustrated in **Figure 1A** [↗](#), which outlines its five-step process. The output of Step 3 is the "events", which is analogous to the output generated by the Nanopolish (v0.14.0) "eventalign" command and can be used as input for downstream models such as m6Anet. An "event" refers to a segment of the raw signal that is aligned to a specific k -mer on a read, along with its associated features such as start and end positions, mean current, standard deviation, and other relevant statistics. Step 4 allows for direct modification prediction at both the site and single-molecule levels. Notably, a key feature of SegPore is the k -mer parameter table, which defines the mean and standard deviation for each k -mer in either an unmodified or modified state. During the training phase, Steps 3~5 are iterated multiple times to stabilize the parameters in the k -mer table, which are subsequently fixed and applied for modification prediction on the test data. A detailed description of the model is provided in Supplementary Note 1. Unless otherwise noted, the following analysis focuses on RNA002 data, using 5-mers rather than k -mers.

Preprocessing

We begin by performing basecalling on the input fast5 file using Guppy (v6.0.1), which converts the raw signal data into ribonucleotide sequences. Next, we align the basecalled sequences to the reference genome using Minimap2 (v2.28-r1209) [\(15 ↗\)](#), generating a mapping between the reads and the reference sequences. Nanopolish provides two independent commands: "polya" and "eventalign". The "polya" command identifies the adapter, poly(A) tail, and transcript region in the raw signal, which we refer to as the poly(A) detection results. The raw signal segment corresponding to the poly(A) tail is used to standardize the raw signal for each read. The "eventalign" command aligns the raw signal to a reference sequence, assigning a signal segment to individual k -mers in the reference. It also computes summary statistics (e.g., mean, standard deviation) from the signal segment for each k -mer. Each k -mer together with its corresponding

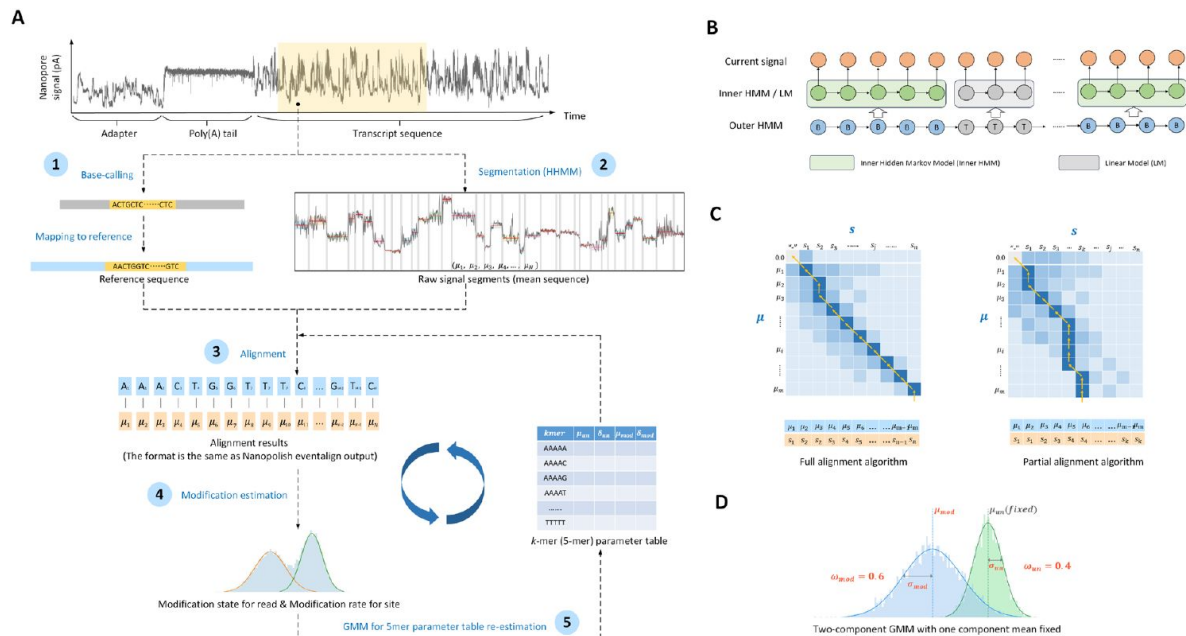


Figure 1.

SegPore workflow.

(A) General workflow. The workflow consists of five steps: (1) First, raw current signals are basecalled and mapped using Guppy and Minimap2. The raw current signal fragments are paired with the corresponding reference RNA sequence fragments using Nanopolish. (2) Next, the raw current signal of each read is segmented using a hierarchical hidden Markov model (HHMM), which provides an estimated mean (μ_i) for each segment. (3) These segments are then aligned with the 5-mer list of the reference sequence fragment using a full/partial alignment algorithm, based on a 5-mer parameter table. For example, A_j denotes the base "A" at the j -th position on the reference. In this example, A_1 and A_2 refer to the first and second occurrences of "A" in the reference sequence, respectively. Accordingly, μ_1 and μ_2 are aligned to A_1 , while μ_3 is aligned to A_2 . (4) All signals aligned to the same 5-mer across different genomic locations are pooled together, and a two-component Gaussian Mixture Model (GMM) is used to predict the modification at the site-level or single-molecule level. One component of the GMM represents the unmodified state, while the other represents the modified state. (5) GMM is used to re-estimate the 5-mer parameter table. (B) Hierarchical hidden Markov model (HHMM). The outer HMM segments the current signal into alternating base and transition blocks. The inner HMM approximates the emission probability of a base block by considering neighboring 5-mers. A linear model is used to approximate the emission probability of a transition block. (C) Full/partial alignment algorithms. Rows represent the estimated means of base blocks from the HHMM, and columns represent the 5-mers of the reference sequence. Each 5-mer can be aligned with multiple estimated means from the current signal. (D) Gaussian mixture model (GMM) for estimating modification states. The GMM consists of two components: the green component models the unmodified state of a 5-mer, and the blue component models the modified state. Each component is described by three parameters: mean (μ), standard deviation (σ), and weight (ω).

signal features is termed an event. These event features are then passed into downstream tools such as m6Anet and CHEUI for RNA modification detection. For full transcriptome analysis (Fig. 3), we extract the aligned raw signal segment and reference sequence segment from Nanopolish's events for each read by using the first and last events as start and end points. For *in vitro* transcription (IVT) data with a known reference sequence (Fig. 4), we extract the raw signal segment corresponding to the transcript region for each input read based on Nanopolish's poly(A) detection results.

Due to inherent variability between nanopores in the sequencing device, the baseline levels and standard deviations of k-mer signals can differ across reads, even for the same transcript. To standardize the signal for downstream analyses, we extract the raw current signal segments corresponding to the poly(A) tail of each read. Since the poly(A) tail provides a stable reference, we standardize the raw current signals for each read, ensuring that the mean and standard deviation are consistent across the poly(A) tail region. This step is crucial for reducing variability between different reads and ensuring more accurate signal segmentation and modification prediction in subsequent steps. See Section 3 of Supplementary Note 1 for more details.

Signal segmentation via hierarchical Hidden Markov model

The RNA translocation hypothesis (see details in the first section of *Results*) naturally leads to the use of a hierarchical Hidden Markov Model (HHMM) to segment the raw current signal. As shown in Figure 1B, the HHMM consists of two layers. The outer HMM divides the raw signal into alternating base and transition blocks, represented by hidden states “B” (base) and “T” (transition). Within each base block, the inner HMM models the current signal at a more granular level.

The inner HMM includes four hidden states: “prev,” “next,” “curr,” and “noise.” These correspond to the previous, next, and current 5-mer in the pore, while “noise” refers to random fluctuations. Each raw current measurement is emitted from one of these hidden states, providing a detailed model of the signal within the base blocks. A linear model with a large absolute slope is used to represent sharp changes in the transition blocks.

To segment the signal, we first model the likelihood of the HHMM. Given the raw current signal $\mathbf{y}$ of a read, the hidden states of the outer hidden HMM are denoted by $\mathbf{g}$. $\mathbf{y}$ and $\mathbf{g}$ are divided into $2K + 1$ blocks $\mathbf{c}$, where $\mathbf{y}^{(k)}$, $\mathbf{g}^{(k)}$ correspond to k th block and $\mathbf{c} = (c_1, c_2, \dots, c_K, \dots, c_{2K+1})$, $c_k \in \{B, T\}$. Blocks with odd indices ($k = 1, 3, 5, \dots, 2K + 1$) are base blocks, while those with even indices ($k = 2, 4, \dots, 2K$) are transition blocks. The likelihood of the HHMM is given by

$$p(\mathbf{y}, \mathbf{g}) = p(\mathbf{y}|\mathbf{g})p(\mathbf{g}) \quad (1)$$

$$= p(\mathbf{y}|\mathbf{g})\{\pi_{g_1}^{outer} \prod_{i=2}^N T_{g_{i-1}g_i}^{outer}\} \quad (2)$$

$$= \{\prod_{i=1}^N p(y_i|g_i)\}\{\pi_{g_1}^{outer} \prod_{i=2}^N T_{g_{i-1}g_i}^{outer}\} \quad (3)$$

$$= \{\prod_{k=0}^K p(\mathbf{y}^{(2k+1)}|c_{2k+1} = B)\}\{\prod_{k=1}^K p(\mathbf{y}^{(2k)}|c_{2k} = T)\}\{\pi_{g_1}^{outer} \prod_{i=2}^N T_{g_{i-1}g_i}^{outer}\} \quad (4)$$

where $T_{g_{i-1}g_i}^{outer}$ is the transition matrix of the outer HMM and $\pi_{g_1}^{outer}$ is the probability for the first hidden state. The first part of the Eq. 2 are emission probabilities, and the second part are the transition probabilities. It is not possible to directly compute the emission probabilities of the outer HMM (Eq. 3) since there exist dependencies for the current signal measurements within a base or transition block. Therefore, we use the inner HMM and linear model (Eq. 4) to handle the dependencies and approximate emission probabilities.

The inner HMM models transitions between the “prev,” “next,” “curr,” and “noise” states. For the “prev,” “next,” and “curr” states, a Gaussian distribution is used to model the emission probabilities, while a uniform distribution models the “noise” state. The forward-backward algorithm is used to compute the marginal likelihood of the inner HMM, which approximates the emission probabilities for base blocks. For transition blocks, a standard linear model computes the emission probabilities.

For any given g , we can calculate the joint likelihood (Eq. 1). We enumerate different configurations and select the one with the highest likelihood. This process segments the raw current signal into alternating base and transition blocks, where one or more base blocks may correspond to a single 5-mer. Each base block is characterized by its mean and standard deviation, which are used as input for downstream alignment tasks.

Due to the large size of fast5 files (which can reach terabytes), parameter inference for this model is computationally intensive. To address this, we developed a GPU-based inference algorithm that significantly accelerates the process. More details on this algorithm can be found in Supplementary Note 2.

In summary, the HHMM allows us to accurately segment the raw current signal, providing estimates of the mean and variance for each base block, which are crucial for downstream analyses such as RNA modification prediction.

5-mer & 9-mer parameter table

We downloaded the k-mer models “r9.4_180mv_70bps_5-mer_RNA” from the ONT GitHub repository (https://github.com/nanoporetech/kmer_models) for RNA002 data. The columns labeled “level_mean” and “level_stdv” in these models were used as the mean and standard deviation values for the unmodified 5-mers. These values serve as the initial parameters in the 5-mer parameter table for SegPore, which we refer to as the *ONT 5-mer parameter table*. In the RNA004 data analysis, we obtained the 9-mer parameter table from the source code of f5c (version 1.5). Specifically, we used the array named “rna004_130bps_u_to_t_rna_9mer_template_model_builtin_data” from the following file: <https://raw.githubusercontent.com/hasindu2008/f5c/refs/heads/master/src/model.h> (accessed on 17 October 2025).

The initialization of the k-mer parameter table is a critical step in SegPore’s workflow. By leveraging ONT’s established k-mer models, we ensure that the initial estimates for unmodified k-mers are grounded in empirical data. These initial estimates are refined through SegPore’s iterative parameter estimation process, enabling the model to accurately differentiate between modified and unmodified k-mers during segmentation and modification prediction tasks. The refined k-mer parameters also provide a foundation for downstream analysis, such as alignment and m6A identification.

Alignment of raw signal segment with reference sequence

After segmenting the raw current signal of a read into base and transition blocks using HHMM, we align the means of base blocks with the 5-mer list of the reference sequence.

The alignment process involves three main cases:

1. Base block matching with a 5-mer: In this case, a base block aligns with a 5-mer, which is modeled by a Gaussian distribution. The 5-mer may exist in either an unmodified or modified state, and the corresponding Gaussian parameters are retrieved from the 5-mer parameter table.

2. Base block matching with an insertion: Here, the base block aligns with an indel (“-”), indicating an inserted nucleotide in the read.
3. Deletion in the read: In this case, an indel (0.0) matches with a 5-mer, representing a deleted nucleotide in the read.

The alignment score function models these matching cases as follows. For the first case (a base block matching a 5-mer), we calculate the probability that the base block mean is sampled from either the unmodified or modified 5-mer’s Gaussian distribution. The larger of the two probabilities is used as the match score. For the second and third cases, the alignment is treated as noise, and a fixed uniform distribution is used to calculate the match score.

An important distinction from classical global alignment algorithms (Needleman–Wunsch algorithm) is that one or multiple base blocks may align with a single 5-mer. Given the base block means $\mu = (\mu_1, \mu_2, \dots, \mu_i, \dots, \mu_m)$ and 5-mer list $s = (s_1, s_2, \dots, s_j, \dots, s_n)$, we define $(m + 1) \times (n + 1)$ the score matrix as M (Fig. 1C). The first row and column of the matrix are reserved for indels (“0.0” and “-”), representing insertions or deletions in the base blocks or 5-mers, respectively.

We denote the score function by f . The recursion formula of the dynamic programming alignment algorithm is given by

$$M(i, j) = \max \begin{cases} M(i - 1, j - 1) + f(\mu_i, s_j) \\ M(i, j - 1) + f(0.0, s_j) \\ M(i - 1, j) + f(\mu_i, -) \\ M(i - 1, j) + f(\mu_i, s_j) \end{cases} \quad (5)$$

, where f is the score function. It can be seen that we can still align μ_i with s_j after we have aligned μ_{i-1} with s_j , which fulfills the special consideration that one or multiple base blocks might be aligned with one 5-mer.

We implement two types of alignment algorithms (Fig. 1C) based on the score matrix M :

1. Full Alignment Algorithm: This algorithm aligns the full list of base block means with the full list of 5-mers, similar to classical global alignment. It traces back from the $(m + 1, n + 1)$ position in the score matrix.
2. Partial Alignment Algorithm: This aligns the full list of base blocks with the initial part of the 5-mer list, with no indels allowed in the base block means or the 5-mer list. The trace back starts from the maximum value in the last row of the score matrix.

A detailed description of both alignment algorithms is provided in Supplementary Note 1. The output of the alignment algorithm is an alignment that pairs the base blocks with the 5-mers from the reference sequence for each read (Fig. 1C). Base blocks aligned to the same 5-mer are concatenated into a single raw signal segment (referred to as an “event”), from which various features—such as start and end positions, mean current, and standard deviation—are extracted. Detailed derivation of the mean and standard deviation is provided in Section 5.3 in Supplementary Note 1. In the remainder of this paper, we refer to these resulting events as the output of eventalign analysis, which also represents the final output of the segmentation and alignment task.

Modification prediction

After obtaining the eventalign results, we estimate the modification state of each motif using the 5-mer parameter table. Specifically, for each 5-mer, we compare the probability that its mean is sampled from either the modified or unmodified 5-mer’s Gaussian distribution. If the probability under the modified 5-mer distribution is higher, the 5-mer is predicted to be in the modification state, and vice versa for the unmodified state.

To estimate the overall modification rate at a specific genomic location, we pool all reads that map to that location on the reference sequence. The modification rate is calculated as the proportion of reads that are predicted to be in the modification state at that location. A detailed description of the modification state prediction process can be found in Supplementary Note 1.

GMM for 5-mer parameter table re-estimation

To improve alignment accuracy and enhance modification predictions, we use a Gaussian Mixture Model (GMM) to iteratively fine-tune the 5-mer parameter table. As illustrated in **Figure 1A**, the rows of the 5-mer parameter table represent the 5-mers, while the columns provide the mean and standard deviation for both the unmodified and modified states. We denote a 5-mer by s , with its relevant parameters as $\mu_{s,un}$, $\delta_{s,un}$, $\mu_{s,mod}$, $\delta_{s,mod}$.

Using the alignment results from all reads, we collect the base block means aligned to the same 5-mer (denoted by s) across different reads and genomic locations, particularly those with high modification rates. A two-component GMM is then fit to these base blocks. In this process, the mean of the first component is fixed to $\mu_{s,un}$. From the GMM, we update the parameters $\delta_{s,un}$, $\omega_{s,un}$, $\mu_{s,mod}$, $\delta_{s,mod}$ and $\omega_{s,mod}$, where $\omega_{s,un}$, $\omega_{s,mod}$ represent the weights for the unmodified and modified components, respectively. Afterward, we manually adjust the 5-mer parameter table using heuristics to ensure that the modified 5-mer distribution is significantly distinct from the unmodified distribution (see details in Section 7 of Supplementary Note 1).

This re-estimation process is only performed on the training data. The initial 5-mer parameter table is based on the table provided by ONT. After each iteration of updating the table, we rerun the SegPore workflow from the alignment step onward. Typically, the process is repeated three to five times until the 5-mer parameter table stabilizes (when the average change of mean values of all 5-mers is less than $5e-3$). Once a stabilized 5-mer parameter table is estimated from the training data, it is used for RNA modification estimation in the test data without further updates. A more detailed description of the GMM re-estimation process is provided in Section 6 of Supplementary Note 1.

m6A site level benchmark

The HEK293T wild-type (WT) samples were downloaded from the ENA database (accession number PRJEB40872), while the HCT116 samples were obtained from ENA under accession PRJEB44348. The reference sequence (Homo_sapiens.GRCh38.cdna.ncrna_wtChrIs_modified.fa) was downloaded from <https://doi.org/10.5281/zenodo.4587661>. Ground truth data were sourced from Supplementary Data 1 of Pratanwanich, P.N. et al. (21). Fast5 files for the test dataset (mESC WT samples, mESCs_Mettl3_WT_fast5.tar.gz) (22) were retrieved from the NCBI Sequence Read Archive (SRA) under accession SRP166020.

During training, we initialized the 5-mer parameter table using ONT's data. The standard SegPore workflow was executed on the training data (HEK293T WT samples), using the full alignment algorithm. The 5-mer parameter table estimation was iterated five times. For mapping, reads were first aligned to the cDNA and subsequently converted to genomic locations using Ensembl GTF file (GRCh38, v9). The same 5-mer across different genomic locations was pooled together. A 5-mer was considered significantly modified if its read coverage exceeded 1,500 and the distance between the means of the two Gaussian components in the GMM was greater than 5 picoamperes (pA). As a result, modification parameters were specified for ten significant 5-mers, as illustrated in Supplementary Fig. S2A.

With the estimated 5-mer parameter table from the training data, we then ran the SegPore workflow on the test data. The transcript sequences from GENCODE release version M18 were used as the reference sequence for mapping, with the corresponding GTF file (gencode.vM18.chr_patch_hapl_scaff.annotation.gtf) downloaded from GENCODE to convert

transcript locations into genomic coordinates. It is important to note that the 5-mer parameter table was not re-estimated for the test data. Instead, modification states for each read were directly estimated using the fixed 5-mer parameter table. Due to the differences between human (Supplementary Fig. S2A) and mouse (Supplementary Fig. S2B), only six 5-mers were found to have m6A annotations in the test data's ground truth (Supplementary Fig. S2C). For a genomic location to be identified as a true m6A modification site, it had to correspond to one of these six common 5-mers and have a read coverage of greater than 20. SegPore derived the ROC and PR curves for benchmarking based on the modification rate at each genomic location.

In the SegPore+m6Anet analysis, we fine-tuned the m6Anet model using SegPore's eventalign results to demonstrate improved m6A identification. We started with the pre-trained m6Anet model (available at <https://github.com/Goekelab/m6anet>, model version: HCT116_RNA002) and fine-tuned it using the eventalign results from HCT116 samples. SegPore's eventalign output provided the pairing between each genomic location and its corresponding raw signal segment, which allowed us to extract normalized features such as the normalized mean μ_i , standard deviation σ_i , dwell time l_i (number of data points in the event). For genomic location i , m6Anet extracts a feature vector $x_i = \{\mu_{i-1}, \sigma_{i-1}, l_{i-1}, \mu_i, \sigma_i, l_i, \mu_{i+1}, \sigma_{i+1}, l_{i+1}\}$, which was used as input for m6Anet. Feature vectors from a randomly selected 80% of the genomic locations were used for training, while the remaining 20% were set aside for validation. We ran 100 epochs during fine-tuning, selecting the model that performed best on the validation set.

Ground truth data and the performance of other methods (Tombo v1.5.1, Nanom6A v2.0, m6Anet v1.0, and Epinao v1.2.0) on the mESC dataset were provided through personal communications with Prof. Luo Guanzheng, the corresponding author of the benchmark study referenced (23).

m6A single molecule level benchmark

The benchmark IVT data for single molecule m6A identification was downloaded from NCBI-SRA with accession number SRP166020. CHEUI was used for the benchmark. For the ground truth, every A in every read of the IVT_m6A sample is treated as a m6A modification and every A in every read of the IVT_normalA is treated as a normal A. Detailed methods for calculating modification probability at single molecule level were provided in Supplementary Note 1, Section 6.1-6.2.

Results

RNA translocation hypothesis

Accurate segmentation of raw current signals in direct RNA sequencing remains a major challenge, largely because the precise dynamics of RNA translocation through the pore are not fully understood. To address this, we propose a hypothesis that better reflects the physical movement of the RNA molecule. In traditional basecalling algorithms such as Guppy and Albacore, we implicitly assume that the RNA molecule is translocated through the pore by the motor protein in a monotonic fashion, i.e., the RNA is pulled through the pore unidirectionally. In the DNN training process of Guppy and Albacore, we try to align the current signal with the reference RNA sequence. The alignment is unidirectional, which is the source of the implicit monotonic translocating assumption.

Contrary to the conventional assumption of monotonic translocation, the raw current data suggests that the motor protein drives RNA both forward and backward during sequencing. **Figure 2B** illustrates this with several example fragments of DRS raw current signal (21), where each fragment roughly corresponds to three neighboring 5-mers. The raw current signals, as shown in **Figure 2B**, strongly support this hypothesis, with several instances of measured current intensities matching both the previous and next 5-mer's baseline. These repeated patterns,

observed across multiple DRS datasets, provide empirical evidence of the jiggling translocation. Similar patterns are widely observed across the whole data. This suggests that the RNA molecule may move forward and backward while passing through the pore. This observation is also supported by previous reports (24,25), in which the helicase (the motor protein) translocates the DNA strand through the nanopore in a back-and-forth manner. Depending on ATP or ADP binding, the motor protein may translocate the DNA/RNA forward or backward by 0.5~1 nucleotides.

Based on the reported kinetic model (24), we hypothesize that the RNA is translocated through the pore in a jiggling manner. This “jiggling” hypothesis presents a significant departure from the traditionally accepted view of unidirectional RNA translocation and aligns better with recent evidence from studies on DNA translocation (24,25). Incorporating this hypothesis into segmentation algorithms could lead to more accurate predictions of RNA modifications by accounting for the dynamic nature of translocation. On average, the motor protein sequentially translocates 5-mers on the RNA strand forward, and each 5-mer resides in the pore for a short time. During the short period of a single 5-mer, the motor protein may swiftly drive the RNA molecule forward and backward by 0.5~1 nucleotide in the translocation process of the current 5-mer (**Fig. 2A**), which makes the measured current intensity occasionally similar to the previous or the next 5-mer. When the motor protein does not move the RNA molecule, we hypothesize that the RNA molecule undergoes slight thermal fluctuations, causing it to oscillate slightly within the pore and produce a stable current close to its baseline. In contrast, sharp changes in current intensity between consecutive 5-mers define transition blocks, where one 5-mer is replaced by the next.

We also assume that the raw current signal of a read can be segmented into a series of alternating base and transition blocks. In the ideal case, a base block corresponds to the base state where the 5-mer resides in the pore and jiggles between neighboring 5-mers, i.e., the current 5-mer can transiently jump to the previous or the next 5-mer. A transition block corresponds to the transition state between two consecutive base states where one 5-mer translocates to the next 5-mer in the pore. The current signal should be relatively flat in the base blocks, while a sharp change is expected in the transition blocks. One challenge we encountered was the overestimation of transition blocks. This may be due to a base block actually corresponding to a sub-state of a single 5-mer, rather than each base block corresponding to a full 5-mer, leading to inflated transition counts. To address this issue, SegPore’s alignment algorithm was refined to merge multiple base blocks (which may represent sub-states of the same 5-mer) into a single 5-mer, thereby facilitating further analysis.

SegPore Workflow

The SegPore workflow (**Fig. 1**) consists of five key steps: (1) Preprocess fast5 files to pair raw current signal segments with corresponding RNA sequence fragments; (2) Segment each raw current signal using a hierarchical hidden Markov model (HHMM) into base and transition blocks; (3) Align the derived base blocks with the paired RNA sequence; (4) Fit a two-component GMM to estimate the modification state at the single-molecule level or the modification rate at the site level; (5) Use the results from Step (4) to update relevant parameters. Steps (3) to (5) are iterative and continue until the estimated parameters stabilize.

The final outputs of SegPore are the events and modification state predictions. SegPore’s events are similar to the outputs of Nanopolish’s “eventalign” command, in that they pair raw current signal segments with the corresponding RNA reference 5-mers. Each 5-mer is associated with various features, such as start and end positions, mean current, and standard deviation, derived from the paired signal segment. For 5-mers that exhibit one clearly unmodified component and one clearly modified component, SegPore reports the modification rate at each site, as well as the modification state of that site on individual reads.

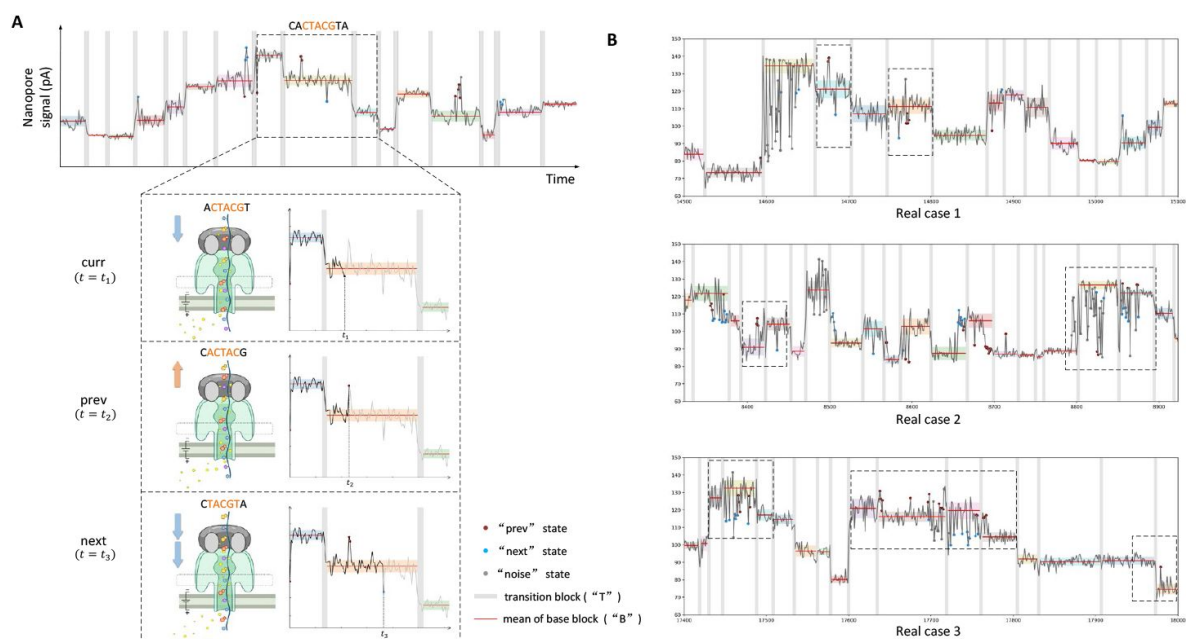


Figure 2.

RNA translocation hypothesis.

(A) Jiggling RNA translocation hypothesis. The top panel shows the raw current signal of Nanopore direct RNA sequencing, with gray areas representing SegPore-estimated transition blocks. We focus on three neighboring 5-mers, considering the central 5-mer (CTACG) as the current 5-mer. The RNA molecule may briefly move forward or backward during the translocation of the current 5-mer. If the RNA molecule is pulled backward, the previous 5-mer is placed in the pore, and the current signal ("prev" state, red dots) resembles the previous 5-mer's baseline (mean and standard deviation highlighted by red lines and shades). If the RNA is pushed forward, the current signal ("next" state, blue dots) is similar to the next 5-mer's baseline. (B) Example raw current signals supporting the jiggling hypothesis. The dashed rectangles highlight base blocks, with red and blue points representing measurements corresponding to the previous and next 5-mer, respectively. Red points align closely with the previous 5-mer's baseline, and blue points match the next 5-mer's baseline, reinforcing the hypothesis that the RNA molecule jiggles between neighboring 5-mers. The raw current signals were extracted from mESC WT samples of the training data in the m6A benchmark experiment.

A key component of SegPore is the 5-mer parameter table, which specifies the mean and standard deviation for each 5-mer in both modified and unmodified states (**Fig. 1A**). Since the peaks (representing modified and unmodified states) are separable for only a subset of 5-mers, SegPore can provide modification parameters for these specific 5-mers. For other 5-mers, modification state predictions are unavailable.

Segmentation benchmark

To evaluate SegPore's performance in raw signal segmentation, we compared SegPore with Nanopolish (v0.14.0) and Tombo (v1.5.1) using three Nanopore direct RNA sequencing (DRS) datasets: two HEK293T datasets (wild type and Mettl3 knockout) (21) and the HCT116 dataset (26). Nanopolish and SegPore employed the “eventalign” method to align 5-mers on each read with their corresponding raw signals, producing the mean and standard deviation (std) of the aligned signal segments. Tombo used the “resquiggle” method to segment the raw signals, but the resulting signals are not reported on the absolute pA scale. To ensure a fair comparison with SegPore, we standardized the segments using the poly(A) tail in the same way as SegPore (See preprocessing section in Materials and Methods).

To benchmark segmentation performance, we used two key metrics (details provided in Supplementary Note 1, Section 8): (1) the log-likelihood of the segment mean, which measures how closely the segment matches ONT's 5-mer parameter table (used as ground truth), and (2) the standard deviation (std) of the segment, where a lower std indicates reduced noise and better segmentation quality. If the raw signal segment aligns correctly with the corresponding 5-mer, its mean should closely match ONT's reference, yielding a high log-likelihood. A lower std of the segment reflects less noise and better performance overall.

As shown in **Table 1**, SegPore consistently achieved the best performance averaged on all 5-mers across all datasets, with the highest log-likelihood and the lowest std values. These results suggest that SegPore provides a more accurate segmentation of the raw signal with significantly reduced noise compared to Nanopolish and Tombo. It is worth noting that the data points corresponding to the transition state between two consecutive 5-mers are not included in the calculation of the standard deviation in SegPore's results in **Table 1**. However, their exclusion does not affect the overall conclusion, as there are on average only ~6 points per event in the transition state (see Supplementary Table S1 for more details).

To provide a more intuitive comparison, the segmentation results for two example raw signal clips are illustrated in Supplementary Fig. S1. These examples demonstrate the clearer, more precise segmentation achieved by SegPore compared to Nanopolish.

To evaluate SegPore's performance on RNA004 data, we compared it with f5c (v1.5) (27) and Uncalled4 (28) (v4.1.0) using three public DRS datasets: the *S. cerevisiae* dataset (29), the curlcake IVT and m6A datasets (30). The RNA002 data provides reference current levels for 5-mers, whereas RNA004 provides reference values for 9-mers, with Uncalled 4 normalizing them to approximately zero mean and unit variance. As there are currently no established poly(A) detection methods available for RNA004, we used f5c to standardize the raw signals of each read prior to segmentation. SegPore was then applied to perform segmentation on the standardized signals. We computed the same benchmarking metrics—average log-likelihood and standard deviation—using the standardized raw signals and the segmentation results from f5c, Uncalled4, and SegPore. The 9-mer parameter table in pA scale for RNA004 data provided by f5c (see Materials and Methods) was used in the analysis. As shown in **Table 2**, SegPore achieved the best overall performance across all datasets, indicating its robustness and suitability for RNA004 data. Moreover, we find that the jiggling hypothesis remains valid for RNA004, as illustrated in Supplementary Fig. S4.

Table 1.

Segmentation benchmark on RNA002 data

Test Dataset	Avg. std ($\hat{\sigma}$) ↓			Avg. log p ($\hat{L}$) ↑		
	Nanopolish	Tombo	SegPore	Nanopolish	Tombo	SegPore
HEK293T(WT)	3.073	4.187	2.736	-2.871	-3.749	-2.778
HEK293T(KO)	2.948	4.204	2.670	-2.856	-3.749	-2.745
HCT116	3.167	4.076	2.872	-2.872	-3.704	-2.746

Table 2.

Segmentation benchmark on RNA004 data

Test Dataset	Avg. std ($\hat{\sigma}$) ↓			Avg. log p ($\hat{L}$) ↑		
	f5c	Uncalled4	SegPore	f5c	Uncalled4	SegPore
<i>S. cerevisiae</i>	3.129	3.970	3.125	-2.541	-3.835	-2.489
curlcake(IVT)	3.424	4.729	3.359	-2.716	-2.904	-2.515
curlcake(m6A)	3.520	4.971	3.392	-3.118	-3.524	-2.599

m6A identification at the site level

We evaluated SegPore's performance in raw signal segmentation and m6A identification using independent public datasets as both training and test data. Since m6A modifications typically occur at DRACH motifs (where D denotes A, G, or U, and H denotes A, C, or U) (13), this study focuses on estimating m6A modifications on these motifs.

To begin, we estimated the 5-mer parameter table for m6A modifications using Nanopore direct RNA sequencing (DRS) data from three wild-type human HEK293T cell samples (21). The fast5 files from all samples were concatenated, and the full SegPore workflow was run to obtain the 5-mer parameter table. The parameter estimation process was iterated five times to ensure stabilization, refining the modification parameters for ten 5-mers where the modification state distribution significantly differs from the unmodified state.

Next, we applied SegPore's segmentation and m6A identification to test data from wild-type mouse embryonic stem cells (mESCs) (23). Given the comparable methods and input data requirements, we benchmarked SegPore against several baseline tools, including Tombo, MINES (31), Nanom6A (32), m6Anet, Epino (33), and CHEUI (20). By default, MINES and Nanom6A use eventalign results generated by Tombo, while m6Anet, Epino, and CHEUI rely on eventalign results produced by Nanopolish. In **Fig. 3C**, "Nanopolish+m6Anet" refers to the default m6Anet pipeline, whereas "SegPore+m6Anet" denotes a configuration in which Nanopolish's eventalign results are replaced with those from SegPore. Based on the output of SegPore eventalign, we fine-tune m6Anet using the HCT116 data at all DRACH motifs, aiming to demonstrate that the performance of SegPore benefits downstream models. Additionally, due to the differences in the availability of ground truth labels for specific k-mer motifs between human and mouse (Supplementary Fig. S2), six shared 5-mers were selected to demonstrate SegPore's performance in modification prediction directly. By utilizing the 5-mer parameter table derived from the training data, SegPore employs a two-component GMM to calculate the modification rates at the selected m6A sites.

SegPore demonstrated improved segmentation compared to Nanopolish. **Figure 3A** shows the eventalign results at an example genomic location with m6A modifications. SegPore's results show a more pronounced bimodal distribution in the raw signal segment mean, indicating clearer separation of modified and unmodified signals. Furthermore, when pooling all reads mapped to m6A-modified locations at the GGACT motif, SegPore exhibited more distinct peaks (**Fig. 3B**), indicating reduced noise and potentially enabling more reliable modification detection.

We evaluated m6A predictions using two approaches: (1) SegPore's segmentation results were fed into m6Anet, referred to as SegPore+m6Anet, which works for all DRACH motifs and (2) direct m6A predictions from SegPore's Gaussian Mixture Model (GMM), which is limited to the six selected 5-mers shown in Supplementary Fig. S2C that exhibit clearly separable modified and unmodified components in the GMM (see Materials and Methods for details).

In terms of m6A identification, SegPore performed strongly on the test data. Using miCLIP2 (34) data as the ground truth, we calculated the area under the curve (AUC) for both the receiver operating characteristic (ROC) and precision-recall (PR) curves. SegPore+m6Anet achieved the best performance with an ROC AUC of 89.0% and a PR AUC of 43.8% (**Fig. 3C**). For six selected m6A motifs, SegPore achieved an ROC AUC of 82.7% and a PR AUC of 38.7%, earning the third best performance compared with deep learning methods m6Anet and CHEUI (**Fig. 3D**). It is noteworthy that SegPore's GMM for m6A estimation is a very simple model, utilizing far fewer parameters than DNN-based methods. Achieving the decent performance with such a simple model is a significant accomplishment. These results highlight SegPore's robust performance in

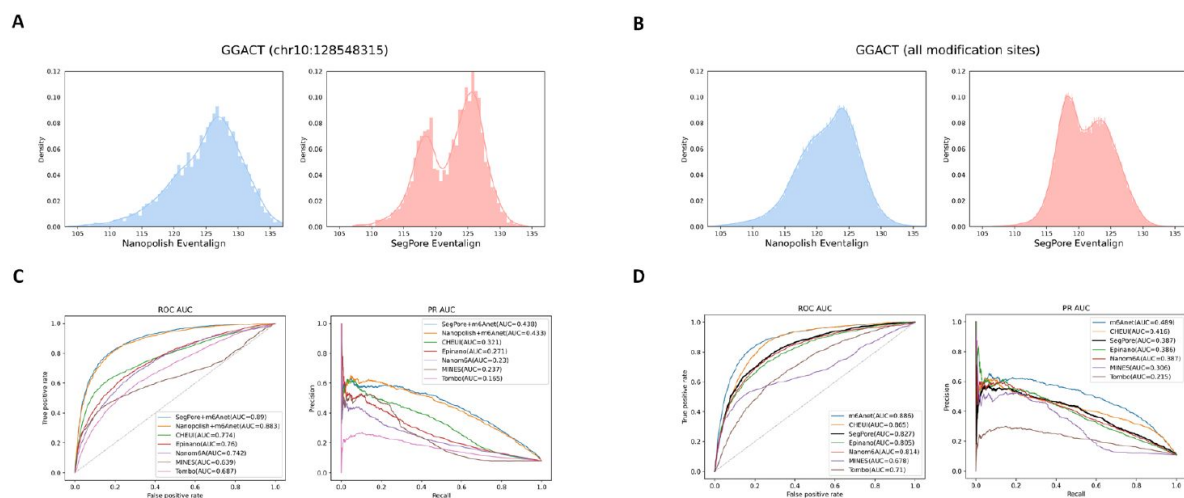


Figure 3.

m6A identification at the site level.

(A) Histogram of the estimated mean from current signals mapped to an example m6A-modified genomic location (chr10:128548315, GGACT) across all reads in the training data, comparing Nanopolish (left) and SegPore (right). The x-axis represents current in picoamperes (pA). (B) Histogram of the estimated mean from current signals mapped to the GGACT motif at all annotated m6A-modified genomic locations in the training data, again comparing Nanopolish (left) and SegPore (right). The x-axis represents current in picoamperes (pA). (C) Site-level benchmark results for m6A identification across all DRACH motifs, showing performance comparisons between SegPore+m6Anet and other methods. (D) Benchmark results for m6A identification on six selected motifs at the site level, comparing SegPore and other baseline methods.

m6A identification. For practical applications, we recommend taking the intersection of m6A sites predicted by SegPore and m6Anet to obtain high-confidence modification sites, while still benefiting from the interpretability provided by SegPore's predictions.

m6A identification at the single molecule level

SegPore naturally identifies m6A modifications at the single-molecule level, which is crucial for understanding the heterogeneity of RNA modifications across individual transcripts. We benchmarked SegPore against CHEUI using an *in vitro* transcription (IVT) dataset containing two samples—one transcribed with m6A and the other with adenine (22). This dataset provides clear ground truth for m6A modifications at the single-molecule level, with all adenosine positions replaced by m6A in the *ivt_m6A* sample, and all adenosines unmodified in the *ivt_normalA* sample. We used 60% of the data for training and 40% for testing, with both SegPore and CHEUI estimating the m6A modification probability at each adenosine site on each read. Based on these probabilities, we calculated the ROC-AUC and PR-AUC by varying the modification probability threshold.

As shown in **Figure 4A**, SegPore outperformed CHEUI on this benchmark dataset, achieving better performance in both PR-AUC (94.7% vs 90.3%) and ROC-AUC (93% vs 89.9%). These results clearly demonstrate SegPore's accuracy and robustness in detecting single-molecule m6A modifications.

Next, we demonstrate SegPore's interpretability through an example comparison of raw signal clips from both *ivt_m6A* and *ivt_normalA* samples (**Fig. 4B**). The raw signal segments (means) are aligned to a randomly selected m6A site as well as its neighbouring sites in the reference sequence. Each line represents an individual read, with pink lines from the *ivt_m6A* sample and gray lines from the *ivt_normalA* sample. SegPore's segmentation clearly distinguishes between m6A and adenosine at the single-molecule level, while Nanopolish's segmentation shows less distinction.

Interestingly, we observed that the position of m6A within a 5-mer can affect the signal intensity. For instance, a clear difference between m6A and adenosine is evident when m6A occupies the fourth position in the 5-mer (e.g., "CGGAC"), but this difference is less pronounced when m6A is in the second position (e.g., "GACTT").

To illustrate the benefits of single-molecule m6A estimation, we present an example gene (ENSMUSG00000003153) from the mESC dataset used in site-level m6A identification. **Figure 4C** shows a heatmap of highly modified genomic locations (modification rate > 0.1), where rows represent reads and columns represent genomic locations. Biclustering reveals six clusters of reads and three clusters of genomic locations.

The heatmap in **Figure 4C** suggests heterogeneity in m6A modification patterns across different reads of the same gene. Biclustering reveals that modifications at g6 are specific to cluster C4, g7 to cluster C5, and g8 to cluster C6, while the first five genomic locations (g1 to g5) show similar modification patterns across all reads. Additionally, high modification rates are observed at the 3rd and 5th positions across the majority of reads. These results suggest that m6A modification patterns can vary significantly even within a single gene. This observation highlights the complexity of RNA modification regulation and underscores the importance of single-molecule resolution for understanding RNA function at a finer scale.

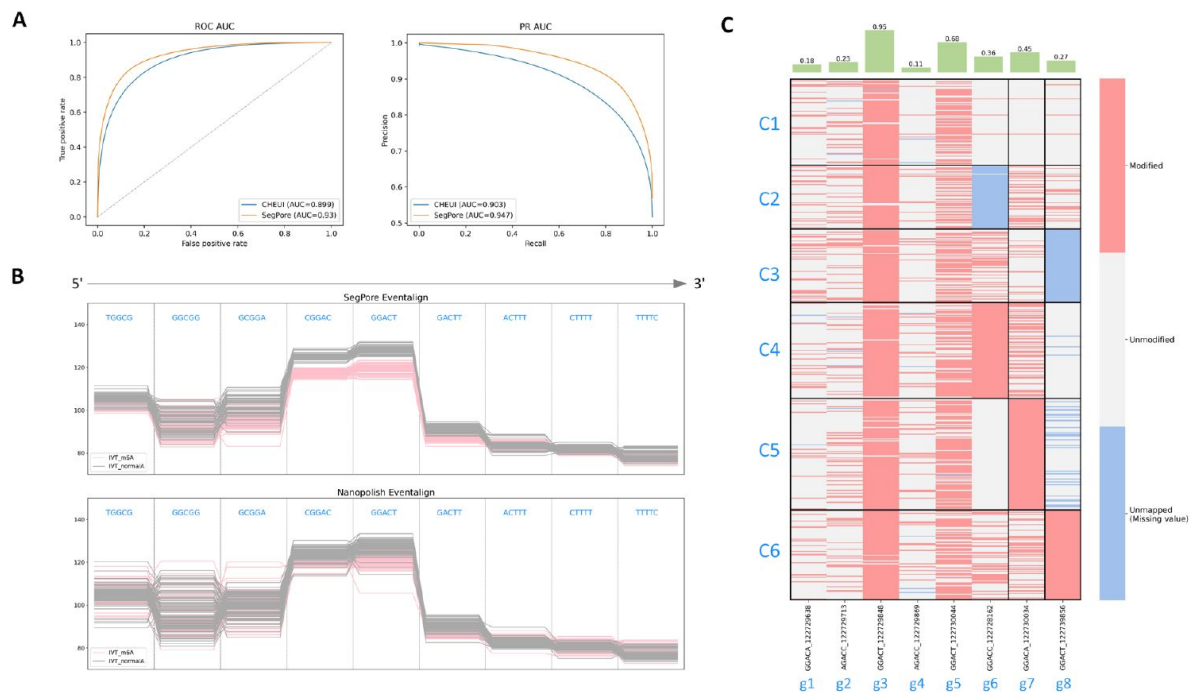


Figure 4.

m6A identification at the single-molecule level.

(A) Benchmark results for single-molecule m6A identification on IVT data. SegPore shows better performance compared to CHEUI in both PR-AUC and ROC-AUC. (B) Comparison of "eventalign" results from SegPore and Nanopolish for five consecutive k-mers. Note that DRS is sequenced from 3' to 5', so the k-mers enters the pore from right to left. A total of 100 reads were randomly sampled from transcript locations A1 (positions 711-719) in both the IVT_normalA and IVT_m6A samples (SRA: SRP166020). Each line represents an individual read, and the y-axis shows the raw signal intensity in picoampere (pA). Pink lines represent the IVT_m6A sample, and gray lines represent the IVT_normalA sample. The k-mers "GCGGA," "CGGAC," "GGACT," "GACTT," and "ACTTT" all contain N6-Methyladenosine (m6A) in the IVT_m6A sample. SegPore's results show clearer separation between m6A and adenosine, especially for "CGGAC" and "GGACT," compared to Nanopolish. (C) The upper panel shows the modification rate for selected genomic locations in the example gene ENSMUSG00000003153. The lower panel displays the modification states of all reads mapped to this gene. The black borders in the heatmap highlight the biclustering results, showing distinct modification patterns across different read clusters labeled C1 through C6.

Discussion

One of the main computational challenges in direct RNA sequencing (DRS) lies in accurately segmenting the raw current signal. We developed a segmentation algorithm that models the jiggling property in the physical process of DRS, resulting in cleaner current signals for m6A identification at both the site and single-molecule levels. Our results demonstrate that SegPore's segmentation enables clear differentiation between m6A-modified and unmodified adenosines. We believe that the de-noised current signals will be beneficial for other downstream tasks, such as the estimation of m5C, pseudouridine, and other RNA modifications. Nevertheless, several open questions remain for future research. In SegPore, we assume a drastic change between two consecutive 5-mers, which may hold for 5-mers with large difference in their current baselines but may not hold for those with small difference. As with other RNA modification estimation methods, SegPore can be affected by misalignment errors, particularly when the baseline signals of adjacent k-mers are similar. These cases may lead to spurious bimodal signal distributions and require careful interpretation. Another key question concerns the physical interpretation of the derived base blocks. Ideally, one base block would correspond to a single 5-mer, but in practice, multiple base blocks often align with one 5-mer. We hypothesize that the HHMM may segment a 5-mer's current signal into multiple base blocks, where the 5-mer oscillates between different sub-states, each characterized by distinct baselines.

Currently, SegPore models only the modification state of the central nucleotide within the 5-mer. However, modifications at other positions may also affect the signal, as shown in **Figure 4B**. Therefore, introducing multiple states for a 5-mer to account for modifications at multiple positions within the same 5-mer could help to improve the performance of the model. This approach, however, would significantly increase model complexity—introducing two states per location results in $2^5 = 32$ modification states per 5-mer, a challenge for simple GMMs. It remains to be investigated how to model multiple modification states properly to improve the performance in RNA modification estimation. While the current two-state assumption simplifies the model, it has proven effective, as demonstrated by improved performance on m6A benchmarks at both site and single-molecule levels.

A key advantage of SegPore is its interpretability, which sets it apart from DNN-based methods. SegPore provides a clearer understanding of RNA modification predictions. A limitation of DNN-based approaches is that they often struggle to differentiate between m6A and adenosine signals, as their intensity levels are quite similar (**Fig. 4B**, Supplementary Fig. S3). This poses a challenge when applying DNN-based methods to new datasets without short read sequencing-based ground truth. In such cases, it is difficult for researchers to confidently determine whether a predicted m6A modification is genuine. SegPore encodes current intensity levels for different 5-mers in a parameter table, where unmodified and modified 5-mers are modeled using two Gaussian distributions. One can generally observe a clear difference in the intensity levels between 5-mers with an m6A and those with adenosine, which makes it easier for a researcher to interpret whether a predicted m6A site is genuine (see Supplementary Fig. S5). A challenge is the relatively small number of 5-mers that show significant changes in their modification states. To improve accuracy, larger training datasets and expanding the scope to 7-mers or 9-mers could help capture more context, potentially revealing more significant baseline changes.

The limited improvement of SegPore combined with m6Anet over Nanopolish+m6Anet in bulk *in vivo* analysis (**Fig. 3**) may be explained by several factors: potential alignment inaccuracies due to pseudogenes or transcript isoforms, the complexity of *in vivo* datasets containing additional RNA modifications (e.g., m5C, m7G) affecting signal baselines, and the fact that m6Anet is specifically trained on events produced by Nanopolish rather than SegPore. Additionally, the lack of a modification-free control (*in vitro* transcribed) sample in the

benchmark dataset makes it difficult to establish true baselines for each k-mer. Despite these limitations, SegPore demonstrates clear improvement in single-molecule m6A identification in IVT data (**Fig. 4** [↗](#)), suggesting it is particularly well suited for *in vitro* transcription data analysis.

Although SegPore provides clear interpretability, there is potential to explore DNN-based models that can directly leverage SegPore's segmentation results. Currently, m6Anet computes features (e.g., mean and standard deviation) from raw signal segments, which are then fed into a neural network for m6A prediction. However, a more direct approach—where raw signal segments are used as input to a DNN—could allow for the extraction of more intricate features that may exist within the signal but are currently underexplored. These high-order features might capture subtle aspects of the raw signal, leading to improved m6A estimation. Since SegPore currently models only the mean and standard deviation, further work involving advanced DNNs could extend beyond this, uncovering finer patterns in the signal that traditional statistical models might miss.

Computation speed is also a concern when processing fast5 files. We addressed this by implementing a GPU-accelerated inference algorithm in SegPore, resulting in a 10-to 20-fold speedup compared to the CPU-based version. We believe that the GPU-implementation will unlock the full potential of SegPore for a wider range of downstream tasks and larger datasets. SegPore's running times on datasets of varying sizes, using a single NVIDIA DGX-1 V100 GPU and one CPU core, are provided in Supplementary Fig. S6.

In summary, we developed a novel software SegPore that considered the conformation changes of motor protein to segment raw current signal of Nanopore direct RNA sequencing. SegPore effectively masks out noise in the raw signal, leading to improved m6A identification at both site and single-molecule levels.

Acknowledgements

We would like to thank Prof. Zhijie Tan from Wuhan University for a useful discussion about the molecule dynamics of Nanopore sequencing, Dr. Dan Zhang from Sichuan University for helpful tutorials about Nanopore analysis workflows. We also thank Prof. Luo Guanzheng for sharing the m6A benchmark baseline results. G.C. and L.C. acknowledge the computational resources provided by the Aalto Science-IT project.

Additional information

Data availability

The data utilized in this study are obtained from publicly available repositories. Details regarding the accession number and data processing can be found in Methods. The resulting data and the source code are hosted on GitHub (<https://github.com/guangzhaocs/SegPore> [↗](#)).

Author contributions

Guangzhao Cheng: Formal analysis, Methodology, Validation, Writing—original draft & editing. Aki Vehtari: Writing—review. Lu Cheng: Conceptualization, Formal analysis, Methodology, Writing—original draft & review.

Funding

This work was supported by Research Council of Finland grants [335858, 358086 to G.C. and L.C.].

Funding

Research Council of Finland (AKA) (335858)

- Guangzhao Cheng
- Lu Cheng

Research Council of Finland (AKA) (358086)

- Guangzhao Cheng
- Lu Cheng

Additional files

Supplementary Figures, Tables, and Notes [↗](#)

References

1. Yankova E., Aspris D., Tzelepis K. (2021) **The N6-methyladenosine RNA modification in acute myeloid leukemia** *Curr Opin Hematol* **28**:80–85 [Google Scholar](#)
2. Prieto M., Folci A., Martin S. (2020) **Post-translational modifications of the Fragile X Mental Retardation Protein in neuronal function and dysfunction** *Mol Psychiatry* **25**:1688–1703 [Google Scholar](#)
3. Bellodi C., McMahon M., Contreras A., Juliano D., Kopmar N., Nakamura T., Maltby D., Burlingame A., Savage S.A., Shimamura A., Ruggero D. (2013) **H/ACA small RNA dysfunctions in disease reveal key roles for noncoding RNA modifications in hematopoietic stem cell differentiation** *Cell Rep* **3**:1493–1502 [Google Scholar](#)
4. Lee H., Bao S., Qian Y., Geula S., Leslie J., Zhang C., Hanna J.H., Ding L. (2019) **Stage-specific requirement for Mettl3-dependent m(6)A mRNA methylation during haematopoietic stem cell differentiation** *Nat Cell Biol* **21**:700–709 [Google Scholar](#)
5. Quin J., Sedmik J., Vukic D., Khan A., Keegan L.P., O’Connell M.A. (2021) **ADAR RNA Modifications, the Epitranscriptome and Innate Immunity** *Trends Biochem Sci* **46**:758–771 [Google Scholar](#)
6. Boccaletto P., Stefaniak F., Ray A., Cappannini A., Mukherjee S., Purta E., Kurkowska M., Shirvanizadeh N., Destefanis E., Groza P., et al. (2022) **MODOMICS: a database of RNA modification pathways. 2021 update** *Nucleic Acids Res* **50**:D231–D235 [Google Scholar](#)
7. Zimna M., Dolata J., Szweykowska-Kulinska Z., Jarmolowski A. (2023) **The expanding role of RNA modifications in plant RNA polymerase II transcripts: highlights and perspectives** *J Exp Bot* **74**:3975–3986 [Google Scholar](#)
8. Ohira T., Suzuki T. (2024) **Transfer RNA modifications and cellular thermotolerance** *Mol Cell* **84**:94–106 [Google Scholar](#)

9. Chen A.Y., Owens M.C., Liu K.F. (2023) **Coordination of RNA modifications in the brain and beyond** *Mol Psychiatry* **28**:2737–2749 [Google Scholar](#)
10. Agris P.F., Narendran A., Sarachan K., Vare V.Y.P., Eruysal E. (2017) **The Importance of Being Modified: The Role of RNA Modifications in Translational Fidelity** *Enzymes* **41**:1–50 [Google Scholar](#)
11. Nance K.D., Meier J.L. (2021) **Modifications in an Emergency: The Role of N1-Methylpseudouridine in COVID-19 Vaccines** *ACS Cent Sci* **7**:748–756 [Google Scholar](#)
12. Meyer K.D., Saletore Y., Zumbo P., Elemento O., Mason C.E., Jaffrey S.R. (2012) **Comprehensive analysis of mRNA methylation reveals enrichment in 3' UTRs and near stop codons** *Cell* **149**:1635–1646 [Google Scholar](#)
13. Linder B., Grozhik A.V., Olarerin-George A.O., Meydan C., Mason C.E., Jaffrey S.R. (2015) **Single-nucleotide-resolution mapping of m6A and m6Am throughout the transcriptome** *Nat Methods* **12**:767–772 [Google Scholar](#)
14. Koh C.W.Q., Goh Y.T., Goh W.S.S. (2019) **Atlas of quantitative single-base-resolution N(6)-methyl-adenine methylomes** *Nat Commun* **10**:5636 [Google Scholar](#)
15. Li H. (2018) **Minimap2: pairwise alignment for nucleotide sequences** *Bioinformatics* **34**:3094–3100 [Google Scholar](#)
16. Loman N.J., Quick J., Simpson J.T. (2015) **A complete bacterial genome assembled de novo using only nanopore sequencing data** *Nat Methods* **12**:733–735 [Google Scholar](#)
17. Simpson J.T., Workman R.E., Zuzarte P.C., David M., Dursi L.J., Timp W. (2017) **Detecting DNA cytosine methylation using nanopore sequencing** *Nat Methods* **14**:407–410 [Google Scholar](#)
18. Stoiber M., Quick J., Egan R., Eun Lee J., Celniker S., Neely R.K., Loman N., Pennacchio L.A., Brown J. (2017) **De novo Identification of DNA Modifications Enabled by Genome-Guided Nanopore Signal Processing** *bioRxiv* [Google Scholar](#)
19. Hendra C., Pratanwanich P.N., Wan Y.K., Goh W.S.S., Thierry A., Goke J. (2022) **Detection of m6A from direct RNA sequencing using a multiple instance learning framework** *Nat Methods* **19**:1590–1598 [Google Scholar](#)
20. Acera Mateos P. A J.S., Ravindran A., Srivastava A., Woodward K., Mahmud S., Kanchi M., Guarnacci M., Xu J. Z W.S.Y., et al. (2024) **Prediction of m6A and m5C at single-molecule resolution reveals a transcriptome-wide co-occurrence of RNA modifications** *Nat Commun* **15**:3899 [Google Scholar](#)
21. Pratanwanich P.N., Yao F., Chen Y., Koh C.W.Q., Wan Y.K., Hendra C., Poon P., Goh Y.T., Yap P.M.L., Chooi J.Y., et al. (2021) **Identification of differential RNA modifications from nanopore direct RNA sequencing with xPore** *Nat Biotechnol* **39**:1394–1402 [Google Scholar](#)
22. Jenjaroenpun P., Wongsurawat T., Wadley T.D., Wassenaar T.M., Liu J., Dai Q., Wanchai V., Akel N.S., Jamshidi-Parsian A., Franco A.T., et al. (2021) **Decoding the epitranscriptional landscape from native RNA sequences** *Nucleic Acids Res* **49**:e7 [Google Scholar](#)
23. Zhong Z.D., Xie Y.Y., Chen H.X., Lan Y.L., Liu X.H., Ji J.Y., Wu F., Jin L., Chen J., Mak D.W., et al. (2023) **Systematic comparison of tools used for m(6)A mapping from nanopore direct RNA sequencing** *Nat Commun* **14**:1906 [Google Scholar](#)

24. Craig J.M., Laszlo A.H., Brinkerhoff H., Derrington I.M., Noakes M.T., Nova I.C., Tickman B.I., Doering K., de Leeuw N.F., Gundlach J.H. (2017) **Revealing dynamics of helicase translocation on single-stranded DNA using high-resolution nanopore tweezers** *Proc Natl Acad Sci U S A* **114**:11932–11937 [Google Scholar](#)
 25. Caldwell C.C., Spies M. (2017) **Helicase SPRNTing through the nanopore** *Proc Natl Acad Sci U S A* **114**:11809–11811 [Google Scholar](#)
 26. Chen Y., Davidson N.M., Wan Y.K., Patel H., Yao F., Low H.M., Hendra C., Watten L., Sim A., Sawyer C., et al. (2021) **A systematic benchmark of Nanopore long read RNA sequencing for transcript level analysis in human cell lines** *bioRxiv* [Google Scholar](#)
 27. Gamaarachchi H., Lam C.W., Jayatilaka G., Samarakoon H., Simpson J.T., Smith M.A., Parameswaran S. (2020) **GPU accelerated adaptive banded event alignment for rapid comparative nanopore signal analysis** *BMC Bioinformatics* **21**:343 [Google Scholar](#)
 28. Kovaka S., Hook P.W., Jenike K.M., Shivakumar V., Morina L.B., Razaghi R., Timp W., Schatz M.C. (2025) **Uncalled4 improves nanopore DNA and RNA modification detection via fast and accurate signal alignment** *Nat Methods* **22**:681–691 [Google Scholar](#)
 29. Watson K.J., Bromley R.E., Dunning Hotopp J.C. (2025) **Duplexed direct RNA sequencing protocol using polyadenylation and polyuridylation** *Microbiol Resour Announc* **14**:e0104124 [Google Scholar](#)
 30. Cruciani S., Delgado-Tejedor A., Pryszcz L.P., Medina R., Llovera L., Novoa E.M. (2025) **De novo basecalling of RNA modifications at single molecule and nucleotide resolution** *Genome Biol* **26**:38 [Google Scholar](#)
 31. Lorenz D.A., Sathe S., Einstein J.M., Yeo G.W. (2020) **Direct RNA sequencing enables m(6)A detection in endogenous transcript isoforms at base-specific resolution** *RNA* **26**:19–28 [Google Scholar](#)
 32. Gao Y., Liu X., Wu B., Wang H., Xi F., Kohnen M.V., Reddy A.S.N., Gu L. (2021) **Quantitative profiling of N(6)-methyladenosine at single-base resolution in stem-differentiating xylem of *Populus trichocarpa* using Nanopore direct RNA sequencing** *Genome Biol* **22**:22 [Google Scholar](#)
 33. Liu H., Begik O., Lucas M.C., Ramirez J.M., Mason C.E., Wiener D., Schwartz S., Mattick J.S., Smith M.A., Novoa E.M. (2019) **Accurate detection of m(6)A RNA modifications in native RNA sequences** *Nat Commun* **10**:4079 [Google Scholar](#)
 34. Kortel N., Ruckle C., Zhou Y., Busch A., Hoch-Kraft P., Sutandy F.X.R., Haase J., Pradhan M., Musheev M., Ostareck D., et al. (2021) **Deep and accurate detection of m6A RNA modifications using miCLIP2 and m6Aboost machine learning** *Nucleic Acids Res* **49**:e92 [Google Scholar](#)
- Pratanwanich et al (2021) **HEK293T** NCBI BioProject ID PRJEB40872 <https://www.ncbi.nlm.nih.gov/bioproject/?term=PRJEB40872>
- Chen, Y., Davidson, N.M., Wan, Y.K., et al (2021) **HCT116** NCBI BioProject ID PRJEB44348 <https://www.ncbi.nlm.nih.gov/bioproject/?term=PRJEB44348>
- Jenjaroenpun et al (2020) **m6A single molecule IVT data** NCBI Sequence Read Archive ID SRP166020 <https://www.ncbi.nlm.nih.gov/sra/?term=SRP166020>

Author information

Guangzhao Cheng

Department of Computer Science, Aalto University, Espoo, Finland

ORCID iD: [0009-0008-6160-3637](https://orcid.org/0009-0008-6160-3637)

Aki Vehtari

Department of Computer Science, Aalto University, Espoo, Finland

ORCID iD: [0000-0003-2164-9469](https://orcid.org/0000-0003-2164-9469)

Lu Cheng

Department of Computer Science, Aalto University, Espoo, Finland, Institute of Biomedicine, University of Eastern Finland, Kuopio, Finland

ORCID iD: [0000-0002-6391-2360](https://orcid.org/0000-0002-6391-2360)

For correspondence: lu.cheng@uef.fi

Editors

Reviewing Editor

Nicolas Altemose

Stanford University, United States of America

Senior Editor

Alan Moses

University of Toronto, Toronto, Canada

Reviewer #1 (Public review):

Summary:

In this manuscript, the authors describe a new computational method (SegPore), which segments the raw signal from nanopore direct RNA-Seq data to improve the identification of RNA modifications. In addition to signal segmentation, SegPore includes a Gaussian Mixture Model approach to differentiate modified and unmodified bases. SegPore uses Nanopolish to define a first segmentation, which is then refined into base and transition blocks. SegPore also includes a modification prediction model that is included in the output. The authors evaluate the segmentation in comparison to Nanopolish and Tombo (RNA002) as well as f5c and Uncalled 4 (RNA004), and they evaluate the impact on m6A RNA modification detection using data with known m6A sites. In comparison to existing methods, SegPore appears to improve the ability to detect m6A, suggesting that this approach could be used to improve the analysis of direct RNA-Seq data.

Strengths:

SegPore address an important problem (signal data segmentation). By refining the signal into transition and base blocks, noise appears to be reduced, leading to improved m6A identification at the site level as well as for single read predictions. The authors provide a fully documented implementation, including a GPU version that reduces run time. The authors provide a detailed methods description, and the approach to refine segments appears to be new.

Reviewer #2 (Public review):

Summary:

The work seeks to improve detection of RNA m6A modifications using Nanopore sequencing through improvements in raw data analysis. These improvements are said to be in the segmentation of the raw data, although the work appears to position the alignment of raw data to the reference sequence and some further processing as part of the segmentation, and result statistics are mostly shown on the 'data-assigned-to-kmer' level.

As such, the title, abstract and introduction stating the improvement of just the 'segmentation' does not seem to match the work the manuscript actually presents, as the wording seems a bit too limited for the work involved.

The work itself shows minor improvements in m6Anet when replacing Nanopolish' eventalign with this new approach, but clear improvements in the distributions of data assigned per kmer. However, these assignments were improved well enough to enable m6A calling from them directly, both at site-level and at read-level.

A large part of the improvements shown appear to stem from the addition of extra, non-base/kmer specific, states in the segmentation/assignment of the raw data, removing a significant portion of what can be considered technical noise for further analysis. Previous methods enforced assignment of (almost) all raw data, forcing a technically optimal alignment that may lead to suboptimal results in downstream processing as datapoints could be assigned to neighbouring kmers instead, while random noise that is assigned to the correct kmer may also lead to errors in modification detection.

For an optimal alignment between the raw signal and the reference sequence, this approach may yield improvements for downstream processing using other tools.

Additionally, the GMM used for calling the m6A modifications provides a useful, simple and understandable logic to explain the reason a modification was called, as opposed to the black models that are nowadays often employed for these types of tasks.

Appraisal:

The authors have shown their methods ability to identify noise in the raw signal and remove their values from the segmentation and alignment, reducing its influences for further analyses. Figures directly comparing the values per kmer do show a visibly improved assignment of raw data per kmer. As a replacement for Nanopolish' eventalign it seems to have a rather limited, but improved effect, on m6Anet results. At the single read level modification modification calling this work does appear to improve upon CHEUI.

<https://doi.org/10.7554/eLife.104618.3.sa2>

Reviewer #3 (Public review):

Summary:

Nucleotide modifications are important regulators of biological function, however, until recently, their study has been limited by the availability of appropriate analytical methods. Oxford Nanopore direct RNA sequencing preserves nucleotide modifications, permitting their study, however many different nucleotide modifications lack an available base-caller to

accurately identify them. Furthermore, existing tools are computationally intensive, and their results can be difficult to interpret.

Cheng et al. present SegPore, a method designed to improve the segmentation of direct RNA sequencing data and boost the accuracy of modified base detection.

Strengths:

This method is well described and has been benchmarked against a range of publicly available base callers that have been designed to detect modified nucleotides.

Comment from the Reviewing Editor:

The authors have provided responses to the weaknesses highlighted previously and the reviewers were not asked to comment. The authors have now requested a Version of Record.

<https://doi.org/10.7554/eLife.104618.3.sa1>

Author response:

The following is the authors' response to the previous reviews

Public Reviews:

Reviewer #1 (Public review):

Summary:

In this manuscript, the authors describe a new computational method (SegPore), which segments the raw signal from nanopore direct RNA-Seq data to improve the identification of RNA modifications. In addition to signal segmentation, SegPore includes a Gaussian Mixture Model approach to differentiate modified and unmodified bases. SegPore uses Nanopolish to define a first segmentation, which is then refined into base and transition blocks. SegPore also includes a modification prediction model that is included in the output. The authors evaluate the segmentation in comparison to Nanopolish and Tombo (RNA002) as well as f5c and Uncalled 4 (RNA004), and they evaluate the impact on m6A RNA modification detection using data with known m6A sites. In comparison to existing methods, SegPore appears to improve the ability to detect m6A, suggesting that this approach could be used to improve the analysis of direct RNA-Seq data.

Strengths:

SegPore address an important problem (signal data segmentation). By refining the signal into transition and base blocks, noise appears to be reduced, leading to improved m6A identification at the site level as well as for single read predictions. The authors provide a fully documented implementation, including a GPU version that reduces run time. The authors provide a detailed methods description, and the approach to refine segments appears to be new.

Weaknesses:

The authors show that SegPore reduces noise compared to other methods, however the improvement in accuracy appears to be relatively small for the task of identifying m6A. To run SegPore, the GPU version is essential, which could limit the application of this method in practice.

As discussed in Paragraph 4 of the Discussion, we acknowledge that the improvement of SegPore combined with m6Anet over Nanopolish+m6Anet in bulk *in vivo* analysis is modest. This outcome is likely influenced by several factors, including alignment inaccuracies caused by pseudogenes or transcript isoforms, the presence of additional RNA modifications that can affect signal baselines, and the fact that m6Anet is specifically trained on Nanopolish-derived events. Additionally, the absence of a modification-free (in vitro transcribed) control sample in the benchmark dataset makes it challenging to establish true k-mer baselines.

Importantly, these challenges do not exist for *in vitro* data, where the signal is cleaner and better defined. As a result, SegPore achieves a clear and substantial improvement at the single-molecule level, demonstrating the strength of its segmentation approach and its potential to significantly enhance downstream analyses. These results indicate that SegPore is particularly well suited for benchmarking and mechanistic studies of RNA modifications under controlled experimental conditions, and they provide a strong foundation for future developments.

We also recognize that the current requirement for GPU acceleration may limit accessibility in some computational environments. To address this, we plan to further optimize SegPore in future versions to support efficient CPU-only execution, thereby broadening its applicability and impact.

Reviewer #2 (Public review):

Summary:

The work seeks to improve detection of RNA m6A modifications using Nanopore sequencing through improvements in raw data analysis. These improvements are said to be in the segmentation of the raw data, although the work appears to position the alignment of raw data to the reference sequence and some further processing as part of the segmentation, and result statistics are mostly shown on the 'data-assigned-to-kmer' level.

As such, the title, abstract and introduction stating the improvement of just the 'segmentation' does not seem to match the work the manuscript actually presents, as the wording seems a bit too limited for the work involved.

The work itself shows minor improvements in m6Anet when replacing Nanopolish' eventalign with this new approach, but clear improvements in the distributions of data assigned per kmer. However, these assignments were improved well enough to enable m6A calling from them directly, both at site-level and at read-level.

A large part of the improvements shown appear to stem from the addition of extra, non-base/kmer specific, states in the segmentation/assignment of the raw data, removing a significant portion of what can be considered technical noise for further analysis. Previous methods enforced assignment of (almost) all raw data, forcing a technically optimal alignment that may lead to suboptimal results in downstream processing as datapoints could be assigned to neighbouring kmers instead, while random noise that is assigned to the correct kmer may also lead to errors in modification detection.

For an optimal alignment between the raw signal and the reference sequence, this approach may yield improvements for downstream processing using other tools.

Additionally, the GMM used for calling the m6A modifications provides a useful, simple and understandable logic to explain the reason a modification was called, as opposed to the black models that are nowadays often employed for these types of tasks.

Weaknesses:

The manuscript suggests the eventual results are improved compared to Nanopolish. While this is believably shown to be true (Table 1), the effect on the use case presented, downstream differentiation between modified and unmodified status on a base/kmer, is likely limited for during downstream modification calling the noisy distributions are often 'good enough'. E.g. Nanopolish uses the main segmentation+alignment for a first alignment and follows up with a form of targeted local realignment/HMM test for modification calling (and for training too), decreasing the need for the near-perfect segmentation+alignment this work attempts to provide. Any tool applying a similar strategy probably largely negates the problems this manuscript aims to improve upon. Should a use-case come up where this downstream optimisation is not an option, SegPore might provide the necessary improvements in raw data alignment.

Thank you for this thoughtful comment. We agree that many current state-of-the-art (SOTA) methods perform well on benchmark datasets, but we believe there is still substantial room for improvement. Most existing benchmarks are based on limited datasets, primarily focusing on DRACH motifs in human and mouse transcriptomes. However, m6A modifications can also occur in non-DRACH motifs, where current models tend to underperform. Furthermore, other RNA modifications, such as pseudouridine, inosine, and m5C, remain less studied, and their detection is likely to benefit from more accurate and informative signal modeling.

It is also important to emphasize that raw signal segmentation and RNA modification detection are fundamentally distinct tasks. SegPore focuses on improving the segmentation step by producing a cleaner and more interpretable signal, which provides a stronger foundation for downstream analyses. Even if RNA modification detection algorithms such as m6Anet can partially compensate for noisy segmentation in specific cases, starting from a more accurate signal alignment can still lead to improved accuracy, robustness, and interpretability—particularly in challenging scenarios such as non-canonical motifs or less characterized modifications.

Scientific progress in this field is often incremental, and foundational improvements can have a significant long-term impact. By enhancing raw signal segmentation, SegPore contributes an essential building block that we expect will enable the development of more accurate and generalizable RNA modification detection algorithms as the community integrates it into more advanced workflows.

Appraisal:

The authors have shown their methods ability to identify noise in the raw signal and remove their values from the segmentation and alignment, reducing its influences for further analyses. Figures directly comparing the values per kmer do show a visibly improved assignment of raw data per kmer. As a replacement for Nanopolish' eventual results it seems to have a rather limited, but improved effect, on m6Anet results. At the single read level modification modification calling this work does appear to improve upon CHEUI.

Impact:

With the current developments for Nanopore based modification calling largely focusing on Artificial Intelligence, Neural Networks and the likes, improvements made in interpretable approaches provide an important alternative that enables deeper understanding of the data rather than providing a tool that plainly answers the question of whether a base is modified or not, without further explanation. The work presented is

best viewed in context of a workflow where one aims to get an optimal alignment between raw signal data and the reference base sequence for further processing. For example, as presented, as a possible replacement for Nanopolish' eventalign. Here it might enable data exploration and downstream modification calling without the need for local realignments or other approaches that re-consider the distribution of raw data around the target motif, such as a 'local' Hidden Markov Model or Neural Networks. These possibilities are useful for a deeper understanding of the data and further tool development for modification detection works beyond m6A calling.

Reviewer #3 (Public review):

Summary:

Nucleotide modifications are important regulators of biological function, however, until recently, their study has been limited by the availability of appropriate analytical methods. Oxford Nanopore direct RNA sequencing preserves nucleotide modifications, permitting their study, however many different nucleotide modifications lack an available base-caller to accurately identify them. Furthermore, existing tools are computationally intensive, and their results can be difficult to interpret.

Cheng et al. present SegPore, a method designed to improve the segmentation of direct RNA sequencing data and boost the accuracy of modified base detection.

Strengths:

This method is well described and has been benchmarked against a range of publicly available base callers that have been designed to detect modified nucleotides.

Weaknesses:

However, the manuscript has a significant drawback in its current version. The most recent nanopore RNA base callers can distinguish between different ribonucleotide modifications, however, SegPore has not been benchmarked against these models.

The manuscript would be strengthened by benchmarking against the rna004_130bps_hac@v5.1.0 and rna004_130bps_sup@v5.1.0 dorado models, which are reported to detect m5C, m6A_DRACH, inosine_m6A and PseU.

A clear demonstration that SegPore also outperforms the newer RNA base caller models will confirm the utility of this method.

Thank you for highlighting this important limitation. While Dorado, the new ONT basecaller, is publicly available and supports modification-aware basecalling, suitable public datasets for benchmarking m5C, inosine, m6A, and PseU detection on RNA004 are currently lacking. Dorado's modification-aware models are trained on ONT's internal data, which is not publicly released. Therefore, it is currently not feasible to directly evaluate or compare SegPore's performance against Dorado for these RNA modifications.

We would also like to emphasize that SegPore's primary contribution lies in raw signal segmentation, which is an upstream and foundational step in the RNA modification detection pipeline. As more publicly available datasets for RNA004 modification detection become accessible, we plan to extend our work to benchmark and integrate SegPore with modification detection tasks on RNA004 data in future studies.

Recommendations for the authors:

Reviewer #2 (Recommendations for the authors):

Comments based on Author Response

"However, it is valid to compare them on the segmentation task, where SegPore exhibits better performance (Table 1)."

This dodges the point of the actual use case of this approach, as Nanopolish indeed does not support calling modifications for this kind of data, but the general approach it uses might, if adapted for this data, nullify the gains made in the examples presented.

We respectfully disagree with the comment that the advantages demonstrated by SegPore could be "nullified". Although SegPore's performance is indeed more modest in *in vivo* datasets, it shows substantially better performance than CHEUI in *in vitro* data, clearly demonstrating that improved segmentation directly contributes to more accurate RNA modification estimation.

It is worth noting that CHEUI relies on Nanopolish's segmentation results for m6A detection. Despite this, SegPore outperforms CHEUI, further supporting the conclusion that segmentation quality has a meaningful impact on downstream modification calling.

In conclusion, based on our current experimental results, SegPore is particularly well suited for RNA modification analysis from *in vitro* transcribed data, where its improved segmentation provides a clear advantage over existing methods.

Further comments

(2) "(2) Page 3 employ models like Hidden Markov Models (HMM) to segment the signal, but they are prone to noise and inaccuracies"

"That's the alignment/calling part, not the segmentation?"

"Current methods, such as Nanopolish, employ models like Hidden Markov Models (HMM) to segment the signal"

I get the impression the word 'segment' has a different meaning in this work than what I'm used to based on my knowledge around Nanopolish and Tombo, see the deeper code examples further down below.

Additionally, in Nanopolish there is a clear segmentation step (or event detection) without any HMM, then a sort of dynamic timewarping step that aligns the segments and re-combines some segments into a single segment where necessary afterwards. I believe the HMM in Nanopolish is not used at all unless modification calling, but if you can point out otherwise I'm open for proof.

Now I believe it is the meaning of 'segmenting the signal' that confuses me, and now the clarification makes it a bit odd as well:

"Nanopolish and Tombo align the raw signal to the reference sequence to determine which portion of the signal corresponds to each k-mer. We define this process as the segmentation task, referred to as "eventalign" in Nanopolish."

So now it's clearly stated the raw signal is being 'aligned' and then the process is suddenly defined as the 'segmentation task', and again referred to as "eventalign". Why is it not referred to as the 'alignment task' instead?

I understand the segmentation and alignment parts are closely connected but to me, it seems this work picks the wrong word for the problem being solved.

"Unlike Nanopolish and Tombo, which directly align the raw signal to the reference sequence,..."

Looking at their code, I believe both Nanopolish and Tombo actually do segment the data first (or "event detection"), then they align the segments/events they found, and finally multiple events aligned to the same section are merged. See for yourself:

Nanopolish:

https://github.com/jts/nanopolish/blob/master/src/nanopolish_squiggle_read.cpp

Line 233:

cpp

trim_and_segment_raw(fast5_data.rt, trim_start, trim_end, varseg_chunk, varseg_thresh);

*event_table et = detect_events(fast5_data.rt, *ed_params);*

Line 270:

cpp

// align events to the basecalled read

*std::vector event_alignment = adaptive_banded_simple_event_align(*this, *this->base_model[strand_idx], read_sequence);*

Where event detection is further defined at line 268 here:

https://github.com/jts/nanopolish/blob/master/src/thirdparty/scrappie/event_detection.c

Tombo:

<https://github.com/nanoporetech/tombo/blob/master/tombo/resquiggle.py>

line 1162 and onwards shows a 'segment_signal' call and the results are used in a 'find_adaptive_base_assignment' call, where 'segment_signal' starting at line 1057 tries to find where the signal jumps from a series of similar values to another (start of a base change in the pore), stored in 'valid_cpts', and the 'find_adaptive_base_assignment' tries to align the resulting segment values to the expected series of values:

python

valid_cpts, norm_signal, new_scale_values = segment_signal(

map_res, num_events, rsqgl_params, outlier_thresh, const_scale)

event_means = ts.compute_base_means(norm_signal, valid_cpts)

dp_res = find_adaptive_base_assignment(

valid_cpts, event_means, rsqgl_params, std_ref, map_res.genome_seq,

start_clip_bases=map_res.start_clip_bases,

seq_samp_type=seq_samp_type, reg_id=map_res.align_info.ID)

These implementations are also why I find the choice of words for what is segmentation and what is alignment a bit confusing in this work, as both Tombo and Nanopolish do a similar, clear segmentation step (or an "event detection" step), followed by the alignment

of the segments they determined. The terminology in this work appears to deviate from these.

We thank the reviewer for the detailed comments!

First of all, we sincerely apologize for our earlier misunderstanding regarding how Nanopolish and Tombo operate. Based on a closer examination of their source codes, we now recognize that both tools indeed include a segmentation step based on change-point detection methods, after which the resulting segments are aligned to the reference sequence. We have revised the relevant text in the manuscript accordingly:

- “Current methods, such as Nanopolish, employ change-point detection methods to segment the signal and use dynamic programming methods and HMM to align the derived segments to the reference sequence,”
- “We define this process as the segmentation and alignment task (abbreviated as the segmentation task), which is referred to as “eventalign” in Nanopolish.”
- “In SegPore, we segment the raw signal into small fragments using a Hierarchical Hidden Markov Model (HHMM) and align the mean values of these fragments to the reference, where each fragment corresponds to a sub-state of a k-mer. By contrast, Nanopolish and Tombo use change-point-based methods to segment the signal and employ dynamic programming approaches together with profile HMMs to align the resulting segments to the reference sequence.”

Regarding terminology, we originally borrowed the term “segmentation” from speech processing, where it refers to dividing continuous audio signals into meaningful units. In the context of nanopore signal analysis, segmentation and alignment are often tightly coupled steps. Because of this and because our initial focus was on methodological development rather than terminology, we used the term “segmentation task” to describe the combined process of signal segmentation and alignment.

However, we now recognize that this terminology may cause confusion. Changing every instance of “segmentation” to “segmentation and alignment” or “alignment” would require substantial rewriting of the manuscript. Therefore, in this revision, we have clearly defined “segmentation task” as referring to the combined process of segmentation and alignment. We apologize for any earlier confusion and will adopt the term “alignment” in future work for greater clarity.

(3) I think I do understand the meaning, but I do not understand the relevance of the A_j bit in the last sentence. What is it used for?

Based on the response and another close look at Fig1, it turns out the j refers to extremely small numbers 1 and 2 in step 3. You may want to improve readability for these.

Thank you for the suggestion. We have added subscripts to all nucleotides in the reference sequence in Figure 1A and revised the legend to clarify the notation and improve readability. Specifically, we now include the following explanation:

“For example, A_j denotes the base ‘A’ at the j -th position on the reference sequence. In this example, A_1 and A_2 refer to the first and second occurrences of ‘A’ in the reference sequence, respectively. Accordingly, μ_1 and μ_2 are aligned to A_1 , while μ_3 is aligned to A_2 ”.

(6) “We chose to use the poly(A) tail for normalization because it is sequence-invariant- i.e., all poly(A) tails consist of identical k-mers, unlike transcript sequences which vary in composition. In contrast, using the transcript region for normalization can introduce

biases: for instance, reads with more diverse k-mers (having inherently broader signal distributions) would be forced to match the variance of reads with more uniform k-mers, potentially distorting the baseline across k-mers."

While the next part states there was a benchmark showing SegPore still works without this normalization, I think this answer does not touch upon the underlying issue I'm trying to point out here.

- The biases mentioned here due to a more diverse (or different) subsets of k-mers in a read indeed affects the variance of the signal overall.

- As I pointed out in my earlier remark here, this can be resolved using an approach of 'general normalization', 'mapping to expected signal', 'theil-sen fitting of scale and offset', 're-mapping to expected signal', as Tombo and Nanopolish have implemented.

- Alternatively, one could use the reference sequence (using the read mapping information) and base the expected signal mean and standard deviation on that instead.

- The polyA tail stability as an indicator for the variation in the rest of the signal seems a questionable assumption to me. A 'noisy' pore could introduce a large standard deviation using the polyA tail without increasing the deviations on the signal induced by the variety of k-mers, rather it would be representative for the deviations measured within a single k-mer segment. I thought this possible discrepancy is to be expected from a worn out pore, hence I'd imagine reads sequenced later in a run to provide worse results using this method.

In the current version it is not the statement that is unclear, it is the underlying assumption of how this works that I question.

We thank the reviewer for raising this important point and for the insightful discussion. Our choice of using the poly(A) tail for normalization is based on the working hypothesis that the poly(A) signal reflects overall pore-level variability and provides a stable reference for signal scaling. We find this to be a practical and effective approach in most experimental settings.

We agree that more sophisticated strategies, such as "general normalization" or iterative fitting to the expected signal (as implemented in Tombo and Nanopolish), could in principle generate a "better" normalization. However, these approaches are significantly more challenging to implement in practice. This is because signal normalization and alignment are mutually dependent processes: baseline estimates for k-mers influence alignment accuracy, while alignment accuracy, in turn, affects baseline calculation. This interdependence becomes even more complex in the presence of RNA modifications, which alter signal distributions and further confound model fitting.

It is worth noting that this limitation is already evident in our results. As shown in Figure 4B (first and second k-mers), Nanopolish produces more dispersed baselines than SegPore, even for these unmodified k-mers, suggesting inherent limitations in its normalization strategy. Ideally, baselines for the same k-mer should remain highly consistent across different reads.

In contrast, poly(A)-based normalization offers a simpler and more robust solution that avoids this circular dependency. Because poly(A) sequences are compositionally homogeneous, they enable reliable estimation of scaling parameters without assumptions about k-mer composition or modification state. Regarding the reviewer's concern about pore instability, we mitigate this issue by including only high-quality, confidently mapped reads in our analysis, which reduces the likelihood of incorporating signals from degraded or "noisy" pores.

We fully agree that exploring more advanced normalization strategies is an important direction for future work, and we plan to investigate such approaches as the field progresses.

(8) *"In the remainder of this paper, we refer to these resulting events as the output of eventalign analysis or the segmentation task."*

Picking only one descriptor rather than two alternatives would be easier to follow (and I'd prefer the first).

Thank you for the suggestion. We have revised the sentence to:

"In the remainder of this paper, we refer to these resulting events as the output of eventalign analysis, which also represents the final output of the segmentation and alignment task."

(9) *"Additionally, a complete explanation of how the weighted mean is computed is provided in Section 5.3 of Supplementary Note 1. It is derived from signal points that are assigned to a given 5mer."*

I believe there's no more mention of a weighted mean, and I don't get any hits when searching for 'weight'. Is that intentional?

We apologize for the misplacement of the formulas. We have updated Section 5.3 of Supplementary Note 1 to clarify the definition of the weighted mean. Because multiple current signal segments may be aligned to a single k -mer, we computed the weighted mean for each k -mer across these segments, where the weight corresponds to the number of data points assigned to "curr" state in each event.

(17) *Response: We revised the sentence to clarify the selection criteria: "For selected 5mers "that exhibit both a clearly unmodified and a clearly" "modified signal component", "SegPore reports the modification rate at each site," "as well as the modification state of that site on individual reads.""*

So is this the same set described on page 13 In 343 or not?

"Due to the differences between human (Supplementary Fig. S2A) and mouse (Supplementary Fig. S2B), only six 5mers were found to have m6A annotations in the test data's ground truth (Supplementary Fig. S2C). For a genomic location to be identified as a true m6A modification site, it had to correspond to one of these six common 5mers and have a read coverage of greater than 20."

I struggle to interpret the 'For selected 5mers' part, as I'm not sure if this is a selection I'm supposed to already know at this point in the text or if it's a set just introduced here. If the latter, removing the word 'selected' would clear it up for me.

We apologize for the confusion. What we mean is that when pooling signals aligned to the same k -mer across different genomic locations and reads, only a subset of k -mers exhibit a bimodal distribution — one peak corresponding to the unmodified state and another to the modified state. Other k -mers show a unimodal distribution, making it impossible to reliably estimate modification levels. We refer to the subset of k -mers that display a bimodal distribution as the "selected" k -mers.

The "selected k -mers" described on page 13, line 343, must additionally have ground truth labels available in both the training and test datasets. There are 10 k -mers with ground truth annotations in the training data and 11 in the test data, and only 6 of these k -mers are shared between the two datasets, therefore only those 6 overlapping k -mers are retained for evaluation. These 6 k -mers satisfy both criteria: (1) exhibiting a bimodal distribution and (2) having ground truth annotations in both training and test sets.

To improve clarity, we have removed the term "selected" from the sentence.

(21) "Tombo used the "resquiggle" method to segment the raw signals, and we standardized the segments using the "poly(A)" tail to ensure a fair comparison "(See" "preprocessing section in Materials and Methods)."

In the Materials and Methods:

"The raw signal segment corresponding to the poly(A) tail is used to standardize the raw signal for each read."

I cannot find more detailed information here on what the standardization does, do you mean to refer to Supplementary Note 1, Section 3 perhaps?

Thank you for pointing this out. Yes, the standardization procedure is described in detail in Supplementary Note 1, Section 3. Tombo itself does not segment and align the raw signal on the absolute pA scale, which can result in very large variance in the derived events if the raw signal is used directly. To ensure a fair comparison, we therefore applied the same preprocessing steps to Tombo's raw signals as we did for SegPore, using only the event boundary information from Tombo while standardizing the signal in the same way.

We have revised the sentence for clarity as follows:

"Tombo used the "resquiggle" method to segment the raw signals, but the resulting signals are not reported on the absolute pA scale. To ensure a fair comparison with SegPore, we standardized the segments using the poly(A) tail in the same way as SegPore (See preprocessing section in Materials and Methods)."

(22A) *The table shown does help showing the benchmark is unlikely to be 'cheated'. However I am suprised to see the Avg std for Nanopolish and Tombo going up instead of down, as I'd expect the transition values to increase the std, and hence, removing them should decrease these values. So why does this table show the opposite?*

I believe this table is not in the main text or the supplement, would it not be a good idea to cover this point somewhere in the work?

Thank you for this insightful comment. In response, we carefully re-examined our analysis and identified a bug in the code related to boundary removal for Nanopolish. We have now corrected this issue and included the updated results in Supplementary Table S1 of the revised manuscript. As shown in the updated table, the average standard deviations decrease after removing the boundary regions for both Nanopolish and Tombo.

We have now included this table in Supplementary Table S1 in the revised manuscript and added the following clarification:

"It is worth noting that the data points corresponding to the transition state between two consecutive 5-mers are not included in the calculation of the standard deviation in SegPore's results in Table 1. However, their exclusion does not affect the overall conclusion, as there are on average only ~6 points per 5-mer in the transition state (see Supplementary Table S1 for more details)."

(22B) *As mentioned in 2), I'm happy there's a clear definition of what is meant but I found the chosen word a bit odd.*

We apologize for the earlier unclear terminology. We now refer to it as the segmentation and alignment task, abbreviated as the segmentation task.

(23) Reading back I can gather that from the text earlier, but the summation of what is being tested is this:

“including Tombo, MINES (31), Nanom6A (32), m6Anet, Epinano (33), and CHEUI (20). ”

next, the identifier "Nanopolish+m6Anet" is, aside from the figure itself, only mentioned in the discussion. Adding a line that explains that "Nanopolish+m6Anet" is the default method of running m6Anet and "SegPore+m6Anet" replaces the Nanopolish part for m6Anet with Segpore, rather than jumping straight to "SegPore+m6Anet", would clarify where this identifier came from.

Thank you for the helpful suggestion. We have added the identifier to the revised manuscript as follows:

“Given their comparable methodologies and input data requirements, we benchmarked SegPore against several baseline tools, including Tombo, MINES (31), Nanom6A (32), m6Anet, Epinano (33), and CHEUI (20). By default, MINES and Nanom6A use eventalign results generated by Tombo, while m6Anet, Epinano, and CHEUI rely on eventalign results produced by Nanopolish. In Fig. 3C, ‘Nanopolish+m6Anet’ refers to the default m6Anet pipeline, whereas ‘SegPore+m6Anet’ denotes a configuration in which Nanopolish’s eventalign results are replaced with those from SegPore.”

(24) For completeness I'd expect tickmarks and values on the y-axis as well.

Thank you for the suggestion. We have updated Figures 3A and 3B in the revised manuscript to include tick marks and values on the y-axis as requested.

(25) Considering this statement and looking back at figure 3a and 3b, wouldn't this be easier to observe if the histograms/KDE's were plotted with overlap in a single figure?

We appreciate the suggestion. However, we believe that overlaying Figures 3A and 3B into a single panel would make the visualization cluttered and more difficult to interpret.

(29) Please change the sentence in the text to make that clear. As it is written now (while it's the same number of motifs, so one might guess it) it does not seem to refer to that particular set of motifs and could be a new selection of 6 motifs.

We appreciate the suggestion and have revised the sentence for clarity as follows:

“We evaluated m6A predictions using two approaches: (1) SegPore’s segmentation results were fed into m6Anet, referred to as SegPore+m6Anet, which works for all DRACH motifs and (2) direct m6A predictions from SegPore’s Gaussian Mixture Model (GMM), which is limited to the six selected 5-mers shown in Supplementary Fig. S2C that exhibit clearly separable modified and unmodified components in the GMM (see Materials and Methods for details).”

(31) I think we have a different interpretation of the word 'leverage', or perhaps what it applies to. I'd say it leverages the jiggling if there's new information drawn from the jiggling behaviour. It's taking it into account if it filters for it. The HHMM as far as I understand tries to identify the jiggles, and ignore their values for the segmentation etc. So while one might see this as an approach that "leverages the hypothesis", I don't see how this HHMM "leverages the jiggling property" itself.

Thank you for the helpful suggestion. We have replaced the word “leverages” with “models” in the revised manuscript.

New points

pg6ln166: "...we extract the aligned raw signal segment and reference sequence segment from Nanopolish's events [...] we extract the raw signal segment corresponding to the transcript region for each input read based on Nanopolish's poly(A) detection results."

It is not clear as to why this different approach is applied for these two cases in this part of the text.

Thank you for pointing this out. The two approaches refer to different preprocessing strategies for in vivo and in vitro data.

For in vivo data, a large proportion of reads do not span the full-length transcript and often map only to a portion of the reference sequence. Moreover, because a single gene can generate multiple transcript isoforms, a read may align equally well to several possible transcripts. Therefore, we extract only the raw signal segment that corresponds to the mapped portion of the transcript for each read.

In contrast, for in vitro data, the transcript sequence is known precisely. As a result, we can directly extract all raw signals following the poly(A) tail and align them to the complete reference sequence.

pg10ln259: An important distinction from classical global alignment algorithms is that one or multiple base blocks may align with a single 5mer."

If there was usually a 1:1 mapping the alignment algorithm would be more or less a direct match, so I think the multiple blocks aligning to a 5mer thing is actually quite common.

Thank you for the comment. The "classical global alignment algorithm" here refers to the Needleman–Wunsch algorithm used for sequence alignment. Our intention was to highlight the conceptual difference between traditional sequence alignment and nanopore signal alignment. In classical sequence alignment, each base typically aligns to a single position in the reference. In contrast, in nanopore signal alignment, one or multiple signal segments — corresponding to varying dwell times of the motor protein — can align to a single 5-mer.

We have revised the sentence as follows:

"An important distinction from classical global alignment algorithms (Needleman–Wunsch algorithm)....."

pg13ln356: "dwell time" is not defined or used before, I guess it's effectively the number of raw samples per segment but this should be clarified.

Thank you for pointing this out. We have now added a clear definition of dwell time in the text as follows:

"such as the normalized mean μ_i , standard deviation σ_i , dwell time l_i (number of data points in the event)."

pg13ln358: "Feature vectors from 80% of the genomic locations were used for training, while the remaining 20% were set aside for validation."

I assume these are selected randomly but this is not explicitly stated here and should be.

Yes, they are randomly selected. We have revised the sentence as follows:

“Feature vectors from a randomly selected 80% of the genomic locations were used for training, while the remaining 20% were set aside for validation.”

pg18ln488: The manuscript now evaluates RNA004 and compares against f5c and Uncalled4. It mentions the differences between RNA004 and RNA002, namely kmer size and current levels, but does not explain where the starting reference model values for the RNA004 model come from: In pg18ln492 they state "RNA004 provides reference values for 9mers", then later they seem to use a 5mer parameter table (pg19ln508), are they re-using the same table from RNA002 or did they create a 5mer table from the 9mer reference table?

We apologize for the confusion. The reference model table for RNA004 9-mers is obtained from f5c (the array named ‘rna004_130bps_u_to_t_rna_9mer_template_model_builtin_data’ in <https://raw.githubusercontent.com/hasindu2008/f5c/refs/heads/master/src/model.h>).

Author response image 1.

```
static float rna004_130bps_u_to_t_rna_9mer_template_model_builtin_data[] = {
    100.595 ,      2.16365 ,      //      AAAAAAAAAA
    101.159 ,      1.87508 ,      //      AAAAAAAAAAC
    100.052 ,      2.40959 ,      //      AAAAAAAAAAG
    101.98 ,       2.13749 ,      //      AAAAAAAAAAT
    101.037 ,      1.92458 ,      //      AAAAAAACA
    101.127 ,      1.7933 ,       //      AAAAAAACC
    101.791 ,      1.97214 ,      //      AAAAAAACG
    101.82 ,       1.81064 ,      //      AAAAAAACT
    99.9802 ,      2.30809 ,      //      AAAAAAAGA
    97.9351 ,      2.93195 ,      //      AAAAAAAGC
```

We have revised the subsection header “5-mer parameter table” in the Method to “5-mer & 9-mer parameter table” to highlight this and added a paragraph about how to obtain the 9-mer parameter table:

“In the RNA004 data analysis (Table 2), we obtained the 9-mer parameter table from the source code of f5c (version 1.5). Specifically, we used the array named ‘rna004_130bps_u_to_t_rna_9mer_template_model_builtin_data’ from the following file: <https://raw.githubusercontent.com/hasindu2008/f5c/refs/heads/master/src/model.h> (accessed on 17 October 2025).”

Also, in page 18 line 195, we added the following sentence:

“The 9-mer parameter table in pA scale for RNA004 data provided by f5c (see Materials and Methods) was used in the analysis.”

pg19ln520: “Additionally, due to the differences of the k-mer motifs between human and mouse (Supplementary Fig. S2), six shared 5mers were selected to demonstrate SegPore’s performance in modification prediction directly.”

“the differences” - in occurrence rates, as I gather from the supplementary figure, but it would be good to explicitly state it in this sentence itself too.

Thank you for the helpful suggestion. We agree that the original sentence was vague. The main reason for selecting only six 5-mers is the difference in the availability of ground truth labels for specific k-mer motifs between human and mouse datasets. We have revised the sentence accordingly:

“Additionally, due to the differences in the availability of ground truth labels for specific k-mer motifs between human and mouse (Supplementary Fig. S2), six shared 5-mers were selected to directly demonstrate SegPore’s performance in modification prediction.”

pg24ln654: “SegPore codes current intensity levels”

“codes” is meant to be “stores” I guess? Perhaps “encodes”?

Thank you for the suggestion. We have now replaced it with “encodes” in the revised manuscript.

Lastly, looking at the feedback from the other reviewers comment:

The ‘HMM’ mentioned in line 184 looks fine to me, the HHMM is 2 HMM’s in a hierarchical setup and the text now refers to one of these HMM layers. If this is to be changed it would need to state the layer (e.g. “the outer HHMM layer”) throughout the text instead.

We agree with this assessment and believe that the term “inner HMM” is accurate in this context, as it correctly refers to one of the two HMM layers within the HHMM structure. Therefore, we have decided to retain the current terminology.

Reviewer #3 (Recommendations for the authors):

I recommend the publication of this manuscript, provided that the following comments are addressed.

Page 5, Preprocessing: You comment that the poly(A) tail provides a stable reference that is crucial for the normalisation of all reads. How would this step handle reads that have interrupted poly(A) tails (e.g. in the case of mRNA vaccines that employ a linker sequence)? Or cell types that express TENT4A/B, which can include transcripts with non-A residues in the poly(A) tail: <https://www.science.org/doi/full/10.1126/science.aam5794>.

It depends on Nanopolish’s ability to reliably detect the poly(A) tail. In general, the poly(A) region produces a long stretch of signals fluctuating around a current level of ~108.9 pA (RNA002) with relatively stable variation, which allows it to be identified and used for normalization.

For in vivo data, if the poly(A) tail is interrupted (e.g., due to non-A residues or linker sequences), two scenarios are possible:

- (1) The poly(A) tail may not be reliably detected, in which case the corresponding read will be excluded from our analysis.
- (2) Alternatively, Nanopolish may still recognize the initial uninterrupted portion of the poly(A) signal, which is typically sufficient in length and stability to be used for signal normalization.

For in vitro data, the poly(A) tails are uninterrupted, so this issue does not arise.

All analyses presented in this study are based exclusively on reads with reliably detected poly(A) tails.

Page 7, 5mer parameter table: r9.4_180mv_70bps_5mer_RNA is an older kmer model (>2 years). How does your method perform with the newer RNA kmer models that do permit the detection of multiple ribonucleotide modifications? Addressing this comment would

be beneficial, however I understand that it would require the generation of new data, as limited RNA004 datasets are available in the public domain.

“r9.4_180mv_70bps_5mer_RNA” is the most widely used k-mer model for RNA002 data. Regarding the newer k-mer models, we believe the reviewer is referring to the “modification basecalling” models available in Dorado, which are specifically designed for RNA004 data. At present, SegPore can perform RNA modification estimation only on RNA002 data, as this is the platform for which suitable training data and ground truth annotations are available. Evaluating SegPore’s performance with the newer RNA004 modification models would require new datasets containing known modification sites generated with RNA004 chemistry. Since such data are currently unavailable, we have not yet been able to assess SegPore under these conditions. This represents an important future direction for extending and validating our method.

The Methods and Results sections contain redundant information -please streamline the information in these sections and reduce the redundancy.

We thank the reviewer for this suggestion and acknowledge that there is some overlap between the Methods and Results sections. However, we feel that removing these parts could compromise the clarity and readability of the manuscript, especially given that Reviewer 2 emphasized the need for clearer explanations. We therefore decided to retain certain methodological descriptions in the Results section to ensure that key steps are understandable without requiring the reader to constantly cross-reference the Methods.

Minor comments

Please be consistent when referring to k-mers and 5-mers (sometimes denoted as 5mers - please change to 5-mers throughout).

We have revised the manuscript to ensure consistency and now use “5-mers” throughout the text.

Introduction

Lines 80 - 112: Please condense this section to roughly half the length (1-2 paragraphs). In general, the results described in the introduction should be very brief, as they are described in full in the results section.

Thank you for the suggestion. We have condensed the original three paragraphs into a single, more concise paragraph as follows:

"SegPore is a novel tool for direct RNA sequencing (DRS) signal segmentation and alignment, designed to overcome key limitations of existing approaches. By explicitly modeling motor protein dynamics during RNA translocation with a Hierarchical Hidden Markov Model (HHMM), SegPore segments the raw signal into small, biologically meaningful fragments, each corresponding to a k-mer sub-state, which substantially reduces noise and improves segmentation accuracy. After segmentation, these fragments are aligned to the reference sequence and concatenated into larger events, analogous to Nanopolish’s “eventalign” output, which serve as the foundation for downstream analyses. Moreover, the “eventalign” results produced by SegPore enhance interpretability in RNA modification estimation. While deep learning-based tools such as m6Anet classify RNA modifications using complex, non-transparent features (see Supplementary Fig. S5), SegPore employs a simple Gaussian Mixture Model (GMM) to distinguish modified from unmodified nucleotides based on baseline current levels. This transparent modeling approach improves confidence in the

predictions and makes SegPore particularly well-suited for biological applications where interpretability is essential."

| *Line 104: Please change "normal adenosine" to "adenosine".*

We have revised the manuscript as requested and replaced all instances of "normal adenosine" with "adenosine" throughout the text.

Materials and Methods

| *Line 176: Please reword "...we standardize the raw current signals across reads, ensuring that the mean and standard deviation of the poly(A) tail are consistent across all reads."*

| *To "...we standardize the raw current signals for each read, ensuring that the mean and standard deviation are consistent across the poly(A) tail region."*

We have changed sentence as requested.

"Since the poly(A) tail provides a stable reference, we standardize the raw current signals for each read, ensuring that the mean and standard deviation are consistent across the poly(A) tail region."

| *Line 182: Please describe the RNA translocation hypothesis, as this is the first mention of it in the text. Also, why is the Hierarchical Hidden Markov model perfect for addressing the RNA translocation hypothesis? Explain more about how the HHMM works and why it is a suitable choice.*

We have revised the sentence as requested:

"The RNA translocation hypothesis (see details in the first section of Results) naturally leads to the use of a hierarchical Hidden Markov Model (HHMM) to segment the raw current signal."

The motivation of the HHMM is explained in detail in the the first section "RNA translocation hypothesis" of Results. As illustrated in Figure 2, the sequencing data suggest that RNA molecules may translocate back and forth (often referred to as jiggling) while passing through the nanopore. This behavior results in complex current fluctuations that are challenging to model with a simple HMM. The HHMM provides a natural framework to address this because it can model signal dynamics at two levels. The outer HMM distinguishes between two major states — base states (where the signal corresponds to a stable sub-state of a k-mer) and transition states (representing transitions from one base state to the next). Within each base state, an inner HMM models finer signal variation using three states — "curr", "prev", and "next" — corresponding to the current k-mer sub-states and its neighboring k-mer sub-states. This hierarchical structure captures both the stable signal patterns and the stochastic translocation behavior, enabling more accurate and biologically meaningful segmentation of the raw current signal.

| *Line 184: do you mean HHMM? Please be consistent throughout the text.*

As explained in the previous response, the HHMM consists of two layers: an outer HMM and an inner HMM. The term "HMM" in line 184 is meant to be read together with "inner" at the end of line 183, forming the phrase "inner HMM." It seems the reviewer may have overlooked this when reading the text.

| *Line 203: please delete: "It is obviously seen that".*

We have removed the phrase “It is obviously seen that” from the sentence as requested. The revised sentence now reads:

“The first part of Eq. 2 represents the emission probabilities, and the second part represents the transition probabilities.”

Line 314, GMM for 5mer parameter table re-estimation: "Typically, the process is repeated three to five times until the 5mer parameter table stabilizes." How is the stabilisation of the 5mer parameter table quantified? What is a reasonable cut-off that would demonstrate adequate stabilisation of the 5mer parameter table? Please add details of this to the text.

We have revised the sentence to clarify the stabilization criterion as follows:

“Typically, the process is repeated three to five times until the 5-mer parameter table stabilizes (when the average change of mean values of all 5-mers is less than $5e-3$).”

Results

Line 377: Please edit to read "Traditional base calling algorithms such as Guppy and Albacore assume that the RNA molecule is translocated unidirectionally through the pore by the motor protein."

We have revised the sentence as:

“In traditional basecalling algorithms such as Guppy and Albacore, we implicitly assume that the RNA molecule is translocated through the pore by the motor protein in a monotonic fashion, i.e., the RNA is pulled through the pore unidirectionally.”

Line 555, m6A identification at the site level: "For six selected m6A motifs, SegPore achieved an ROC AUC of 82.7% and a PR AUC of 38.7%, earning the third best performance compared with deep learning methods m6Anet and CHEUI (Fig. 3D)." So SegPore performs third best of all deep learning methods. Do you recommend its use in conjunction with m6Anet for m6A detection? Please clarify in the text. This will help to guide users to possible best practice uses of your software.

Thank you for the suggestion. We have added a clarification in the revised manuscript to guide users.

“For practical applications, we recommend taking the intersection of m6A sites predicted by SegPore and m6Anet to obtain high-confidence modification sites, while still benefiting from the interpretability provided by SegPore’s predictions.”

Figures.

Figure 1A please refer to poly(A) tail, rather than polyA tail.

We have updated it to poly(A) tail in the revised manuscript.

<https://doi.org/10.7554/eLife.104618.3.sa0>